

Fundamentals of
DATABASE SYSTEM
CONCEPTS

By

A. K. RAJI

Copyright © A. K. RAJI

All Right Reserved

No part of this book may be reproduced, stored in a retrieval system or transmitted in any form or by any means electronics, mechanical, photocopying or otherwise without prior permission of the author and publisher in writing.

First Published in 2010

Published by:

Power Print Ltd. (A Division of Rafmik International)

No 22 Opo-Malu road, Ilorin

08028237916, 080-34947259, 080-56495181

DEDICATION

This book is dedicated to my lovely children Abdus'Samad and Abdul'Quadry.

PREFACE

The book is an introduction to database system concepts. It presents the fundamental concepts of database management system. These concepts include aspects of database design, database languages, and database-system implementation. It also provides a general overview of the nature and purpose of database systems, the entity-relationship model, relational data model, the user-oriented relational languages (SQL), database-design architecture and Microsoft access and describes the influence of the underlying computer system on the database system. The book covers most of the items considered in the current recommended curriculum on database management system for students in Nigeria tertiary institutions and other Computer Training Institutes. Finally, it is believed that the book of this nature will enhance the understanding of all the students in polytechnic and Universities who are required to offer courses in Database Management System.

RAJI A. K.

ACKNOWLEDGEMENT

Surely all praise is for Almighty Allah (S. W. T), the lord of the universe. I thank Him for given me strength and idea for writing this book. He is the Creator and Giver of knowledge, wisdom and idea that make the things possible. I praise Him for His protection and guidance always and seek His help and forgiveness. I also seek refuge in Him from the evil and the wickedness of our own deeds. May the blessings and peace of Almighty Allah be upon His noble prophet Muhammad (SAW), his household, and his companions and all those who follow them in righteous.

My sincere appreciation goes to all those who in one way or the other contributed to the successful production of this book. Thanks also to authors of numerous textbooks, manuals and journals consulted during the course of preparing the write up.

Finally, my profound gratitude goes to my lovely wife and children for their efforts, understanding and prayers always. Thank you all may Almighty God bless you abundantly.

TABLE OF CONTENTS

Dedication

Preface

Acknowledgement

Chapter 1: General Introduction to Computing

Chapter 2: Introduction to Database System

Chapter 3: Entity-Relationship Model

Chapter 4: Relational Model

Chapter 5: Relational Databases

Chapter 6: Database Design Architecture

Chapter 7: Overview of Database Program

Chapter 8: Microsoft Access 2000 and Above

References

CHAPTER ONE GENERAL INTRODUCTION TO COMPUTING

INTRODUCTION

Computing embodies the act, the methods and resources involved in making the computer solve problems, numeric or non-numeric, real or simulated. The focus of this chapter is on computer, its characteristics, its components and how to use it to make it work and how to transform the problems to be solved into a form that could be processed by computer. Computing is an activity, which provides employment for millions of people, directly or indirectly. It affects hundreds of millions of other positively.

In general term computer can be described as an electronic machine that can accept data/information through an input device (e.g. keyboard), store it in its memory and process the data according to instruction given to it and produce an output through an output device (e.g. printer) in a specific format. In its ordinary meaning, a computer is an instrument or machine that is used to compute or calculate. Basically Computer can be described as **I-P-O-** machine.



Computer Science, as a discipline has in recent times, concentrate attention on the study of the digital computer largely because of its wide applications especially in business of data processing and accounting.

DATA AND INFORMATION

The applications of computer are very useful in the area of data processing and cannot be over emphasized. The main reasons of using computer are to process data quickly and efficiently so that information is obtained on time, meaningfully and accurately.

Data can be defined as basic raw facts and figures that are unanalyzed and unorganized and hence make little or no meaning. It can also be described as raw figures or fact of which information is derived or obtained. It is unprocessed information consisting of details relate to business transaction.

An important point to realize is that data, on its own, has no meaning until when some interpretation or process is placed on them. Examples include 70, 65, Yaradua, Shola, Marketing department etc. The smallest unit of derivable data is known as “*data element*”, for example student’s name, Matriculation number, age, sex, nationality, and level etc.

There are three types of data; these are *Alphabetic*, *Numeric* and *Alphanumeric data*. Alphabetic data consist of alphabet A – Z or a – z, examples are fields name, state, nationality sex, etc. Numeric data consist of data fields that have only numbers 0, 1, 2, 3...9, example is field age. While alphanumeric data consist of alphabet, numeric and special symbols. Examples are student’s Matriculation number, address field, vehicle plate number, etc.

Information is the term used to describe meaningful processed data. This is when data is processed into more meaningful and useful form. Information is an important tool and its development, processing and acquisition is the idea behind the entire data processing exercise. Information is obtained after subjecting data to series of processing operations, which converts related groups of meaningless data into a meaningful form. For instance, I am 70 years old, I score 65% in COM 223, Yaradua became the president of Nigeria on 29th May 2007, etc. In other words;

INFORMATION = DATA + PROCESS

INFORMATION = DATA + INTERPRETATION

Furthermore, computer cannot place interpretations on data, only people are capable of interpreting data. The process involves a series of operation to be performed by the computer on the data that are fed into it. These operations may include arithmetic operations (e.g. addition, subtraction, multiplication and division etc) and logical comparison (less than, greater than etc). In summary, information is gotten from data while data is gotten from the events. The different is that data in an organization could be sales figures, cost figures etc, while the information would be well-defined statements that summarize the data in purely meaningful terms as per profits and losses.

CHARACTERISTICS AND USES OF A COMPUTER

Some characteristics of a typical electronics digital computer that are worthy to note include the following:

- i. Electronic in nature, i.e. Data are represented in form of electrical pulses, operations are electronic and basic components are also electronic e.g. integrated circuits.
- ii. It has high degree of accuracy.
- iii. It has high speed, in order of nanoseconds. This is because operations in computer are performed at electronic speed.
- iv. It can store information on a long-time basis.
- v. It has the ability to perform repetitive operations without getting bored.
- vi. Automatic control. Once a job is started, the computer, through the use of the program will carry out the job until is completed.

It is important to state that due to the above characteristics, computer can be used in all human endeavors. But it will be an arduous task to document all areas which computers have been applied in any detail because they are too numerous to list. Nevertheless, some of these areas are: Research and Education, Office Automation, Law, Medicine, Business, Government, Employment, Security Privacy, Weather Forecasting Classroom, Instruction, to mention a few.

Finally, information processing includes tasks that required some measure of intelligence when carried out by a person, but the extent to which computers may be regarded as intelligent is limited at present. For example, Computers cannot take initiatives, respond to unforeseen circumstances or make judgments beyond what it has in its memory.

COMPONENTS OF COMPUTER SYSTEM

A computer is a machine, which, under the control of stored program, automatically accepts and process data, and supplies the results of that processing. The idea of a **system** is very important in the world of computers. A computer is often referred to as a system. Essentially, a **system** is a collection of parts working together to achieve some common goals. A computer is a system, consisting two major **subsystems**, namely, **Hardware** and **Software**.

THE HARDWARE

The computer hardware refers to the physical, mechanical and electrical components and devices that make up the visible computer system. It is the physical components, solid-state and otherwise, which make up a computer. The components of hardware can be subdivided into the following:

- (a) Input Unit
- (b) Control Unit
- (c) Arithmetic and Logical Unit
- (d) Main Storage
- (e) Output Unit
- (f) External Storage Unit

A simple diagram and structure below represent the major components of computer hardware.

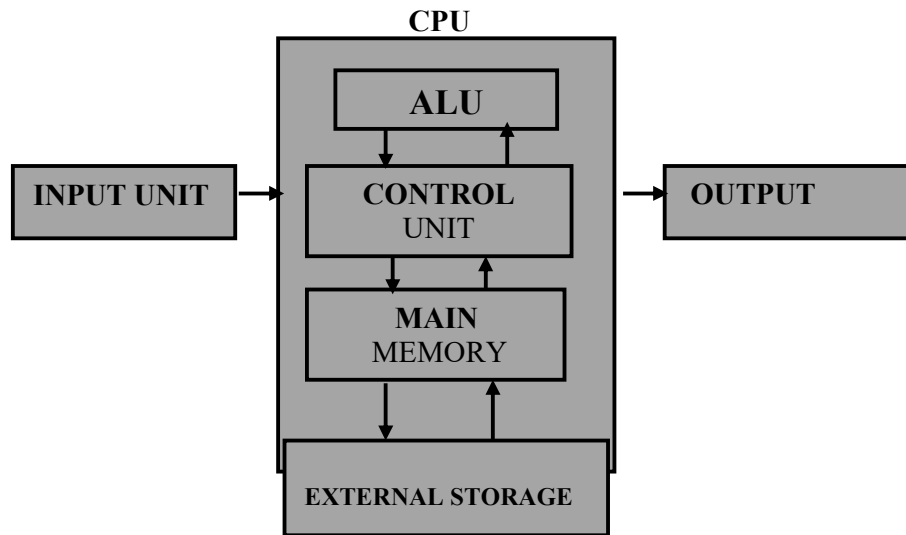


Figure 1.1: The structure of computer Hardware.

Input Unit

Input is the process of getting data / information into a computer from its environment. It provides the medium by which data and instruction are fed into the computer for processing and transmit the data into a series of electrical pulse into the computer primary memory. Examples include; Keyboard, Card Reader, Consoles, Mouse, Optical Character Reader (OCR), Magnetic disk, Magnetic tape, Voice Recognition devices, Mark Sense Reader etc.

Control Unit

The Control Unit ensures the smooth operation of the hardware unit; it fetches and interprets instructions and commands to the unit responsible for executing the instructions. It also instructs the ALU on which arithmetic and logical operation is to be performed.

Arithmetic and Logical Unit

ALU is the part of a computer that is responsible for any arithmetic functions (addition, subtraction, division and multiplication) on data. It has an area called **registers**, where it carries out these functions. In addition to this, it also performs the logical operations such as comparing qualities (e.g. greater than, less than equal to etc.)

Main Memory

It also called **primary storage** and is used for storing data to be processed as well as instructions for processing them. It is fast, wholly electronic and random access memory which is used to store instructions and data. Program can only run or be executed when they are in the main memory and data must be inside the main memory before they can be processed. The features of main memory include: Immediate access, Memory address, Random access and volatility (**RAM** is volatile while **ROM** is non-volatile).

Output Unit

This is the act of getting information out of computer after the processing operations. It provides the medium by which result of computer operations are transmitted to the user. Examples include computer printers, Visual Display Unit (VDU), magnetic storage devices, graph plotter, flash memory etc.

External Storage Unit

This is used to store data/information that are not frequently use by the computer and to store large amount of data/information on a long term basis. They are less expensive than the main memory but access speed is lower than that of the main memory. They can serve as both input and output devices. Examples include magnetic disk, magnetic tape, compact disk etc.

THE SOFTWARE

Software is basically program, which enables the hardware components to operate effectively as well as make it to provide many useful services. They are the programs, which direct the operation of a computer. Programs are sequences of instructions given to computer to solve a given problem or to accomplish a particular task. Software is the collective name used to describe all types of computer programs that can be executed by the system irrespective of what it is designed to do. Without the software, the hardware will not be able to be put into effective use. It has been sometimes said that “Computer without a program is an electronic idiot” simply because it can do nothing constructive or profitable. The term software is usually applied to programs that are written by computer manufacturers and certain software specialists (Computer users). There are two main types of software, namely; System Software and Application Software.

System software

System software is the name given to all the layers of software, which, transform the raw hardware of a computer into a machine, which can be programmed in a straightforward way. These are the programs that are commonly written by the computer manufacturers, which have a direct effect on the control, performance and ease of usage of the computer systems. System program is one of the basic programs required by the computer system to provide services for which the computer is manufactured to users and are usually supplied by the manufacturer along with the computer hardware. Examples of system program include; Operating System, Translators, Database Management System (DBMS) and Utility programs.

a. Operating System

Operating System is system software that consists of collection of program modules that supervises the operation and the use of all system resources to ensure their efficient and judicious management. It serves as an interface (intermediary) between the computer hardware and the computer user. It is a specialized program that is used in starting up the computer. It has the following basic functions:

- Scheduling and loading of program and data for execution.
- Supervises the transfer of data and information and also detects errors arising from user's programs
- It provides some measures of security for program and data.

Examples of OS include DOS/VS, George 3, AOS/VS, PC-DOS, UNIX, NOVEL, and WINDOWS OS.

b. Translator

Language Translators are the programs that translate programs written in non-machine language (High Level or Low Level language) into machine language equivalent. It allows users to write their own programs in language different from the one understood by the computer. There are **four** levels of programming language, which include Machine Language, Low Level Language, High Level Language and Forth Generation Language. Machine language is the only language that can be directly executed by the computer, but non-machine languages (i.e. L.L.L and H.L.L) are not directly executable. In order for programs in these languages be executable, translator is required. Examples include **assembler, compiler, and interpreter.**

c. Database Management System (DBMS)

DBMS can be defined as the complex software that is used for creation, storage, retrieval and maintenance of a database, as well as providing security for the database. A **database** is the collection of related data relevant to the operations of a particular organization. For example data about students, staff and properties of a university will form the operational database of the university. The DBMS maintains the data in the base by adding new records and amending existing records. It provides an interface between the user and the data in the base.

d. Utility Programs

These are the system programs, which provide useful services to the user of the computer by providing facilities for performing common tasks of a routine nature. Examples of these programs are sorting utility, merging utility, text editor and debugging facility

Application software

These are the programs that either written by the computer manufacturer or the individual users to solve their own particular problems which do not have direct effect on the performance and efficiency of the computer system. It can be further sub-divided into **User's application programs and Application packages.**

- (i) **Users application program:** This is a program that belongs to a particular person (i.e. program written by someone) for his own use. This program solves specific problem which interest to the person. Examples include program written to prepare the payroll for over 10,000 workers in an organization, students result, sales analysis, MTN billing charges, scheduling class lectures etc.
- (ii) **Application packages:** These are the ready-made programs designed in a standardized way for application which common to many users. Examples are word-processing, spreadsheet etc.

THE HUMAN-WARE

The human-ware is referred to the people that work with the computer system. It is an inseparable and integral part of the computer. The computer is not automatic, meaning that it has to be initiated by human being. Human being gives the data and the instructions specifying the operations to be carried out. Human-ware includes the programmer, computer operator, data entry operator, system analyst, and computer engineer.

EXERCISE 1

1. Discuss the applications of computer in your locality.
2. With the aid of diagram, describe the functional unit of computer hardware.
3. Computer is an IPO-machine, discuss.
4. Differentiate between data and information, and why do you think they of great importance in computing.
5. Computer system without software is an “electronic idiot”, discuss.
6. Carefully define the term 'package' in this modern age of computing.
7. Discuss the advantages of packages over in-house software.
8. Differentiate between System Software and Application software.

CHAPTER TWO

INTRODUCTION TO DATABASE SYSTEM

INTRODUCTION

The amount of information available to us is literally exploding, and the value of data as an organizational asset is widely recognized. Yet without the ability to manage this vast amount of data, and to quickly find the information that is relevant to a given question, as the amount of information increases, it tends to become a distraction and a liability, rather than an asset. This paradox drives the need for increasingly powerful and flexible data management systems. To get the most out of their large and complex datasets, users must have tools that simplify the tasks of managing the data and extracting useful information in a timely fashion. Otherwise, data can become a liability, with the cost of acquiring it and managing it far exceeding the value that is derived from it.

A **database** is a collection of data, typically describing the activities of one or more related organizations. For example, a polytechnic database might contain information about the following:

- i. Entities such as students, faculty, courses, and lecture rooms.
- ii. Relationships between entities, such as students' enrolment in courses, faculty teaching courses, and the use of rooms for courses.

A database-management system (DBMS) is a collection of interrelated data and a set of programs to access those data. It is software designed to assist in maintaining and utilizing large collections of data, and the need for such systems, as well as their use, is growing rapidly. The collection of data, usually referred to as the **database**, contains information relevant to an enterprise. The primary goal of a DBMS is to provide a way to store and retrieve database information that is both *convenient* and *efficient*.

Database systems are designed to manage large bodies of information. Management of data involves both defining structures for storage of information and providing mechanisms for the manipulation of information. In addition, the database system must ensure the safety of the information stored, despite system crashes or attempts at unauthorized access. If data are to be shared among several users, the system must avoid possible anomalous results.

APPLICATIONS OF DATABASE SYSTEM

The area of database management systems is a microcosm of computer science in general. The issues addressed and the techniques used span a wide spectrum, including languages, object-orientation and other programming paradigms, compilation, operating systems, concurrent programming, data structures, algorithms, theory, parallel and distributed systems, user interfaces, expert systems and artificial intelligence, statistical techniques, and dynamic programming. Databases are widely used. Here are some representative applications:

- i. **Banking:** For customer information, accounts, and loans, and banking transactions.
- ii. **Airlines:** For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner—terminals situated around the world accessed the central database system through phone lines and other data networks.
- iii. **Institutions:** For student information, course registrations, and grades.
- iv. **Credit card transactions:** For purchases on credit cards and generation of monthly statements.

...Fundamentals of Database System Concepts

- v. **Telecommunication:** For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.
- vi. **Finance:** For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds.
- vii. **Sales:** For customer, product, and purchase information.
- viii. **Manufacturing:** For management of supply chain and for tracking production of items in factories, inventories of items in warehouses/stores, and orders for items.
- ix. **Human resources:** For information about employees, salaries, payroll taxes and benefits, and for generation of pay checks.

The last four decades of the twentieth century, use of databases grew in all enterprises. In the early days, very few people interacted directly with database systems, although without realizing it they interacted with databases indirectly-through printed reports such as credit card statements, or through agents such as bank tellers and airline reservation agents. Then automated teller machines came along and let users interact directly with databases. Phone interfaces to computers (interactive voice response systems) also allowed users to deal directly with databases-a caller could dial a number, and press phone keys to enter information or to select alternative options, to find flight arrival/departure times, or to register for courses in a university.

The internet revolution also increased direct user access to databases. For instance, when accessing an online bookstore and browse a book or music collection, you are accessing data stored in a database. When you enter an order online, your order is stored in a database. When you access a bank Web site and retrieve your bank balance and transaction information, the information is retrieved from the bank's database system. When you access a Web site, information about you may be retrieved from a database, to select which advertisements should be shown to you. Finally, data about your Web accesses may be stored in a database. Thus, although user interfaces hide details of access to a database, and most people are not even aware they are dealing with a database, accessing databases forms an essential part of almost everyone's life today.

DATABASE SYSTEMS AND FILE SYSTEMS

File-processing system is supported by a conventional operating system. The system stores permanent records in various files, and it needs different application programs to extract records from, and add records to, the appropriate files. Before database management systems (DBMSs) came along, organizations usually stored information in such systems. Keeping organizational information in a file-processing system has a number of major disadvantages:

- i. **Data redundancy and inconsistency.** Since different programmers create the files and application programs over a long period, the various files are likely to have different formats and the programs may be written in several programming languages. Moreover, the same information may be duplicated in several places (files). For example, the address and telephone number of a particular customer may appear in a file that consists of savings-account records and in a file that consists of checking-account records. This redundancy leads to higher storage and access cost. In addition, it may lead to data inconsistency; that is, the various copies of the same data may no longer agree. For example, a changed customer address may be reflected in savings-account records but not elsewhere in the system.
- ii. **Difficulty in accessing data.** Suppose that one of the bank officers needs to find out the names of all customers who live within a particular postal-code area. The officer asks the data-processing department to generate such a list. Because the designers of the original system did not anticipate this request, there is no application program on hand to meet it. There is, however, an application

...*Fundamentals of Database System Concepts*

program to generate the list of *all* customers. The bank officer has now two choices: either obtain the list of all customers and extract the needed information manually or ask a system programmer to write the necessary application program. Both alternatives are obviously unsatisfactory. Suppose that such a program is written, and that, several days later, the same officer needs to trim that list to include only those customers who have an account balance of ₦10,000 or more. As expected, a program to generate such a list does not exist. Again, the officer has the preceding two options, neither of which is satisfactory. The point here is that conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner. More responsive data-retrieval systems are required for general use.

- iii. **Data isolation.** Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.
- iv. **Integrity problems.** The data values stored in the database must satisfy certain types of consistency constraints. For example, the balance of a bank account may never fall below a prescribed amount (say, ₦1000). Developers enforce these constraints in the system by adding appropriate code in the various application programs. However, when new constraints are added, it is difficult to change the programs to enforce them. The problem is compounded when constraints involve several data items from different files.
- v. **Atomicity problems.** A computer system, like any other mechanical or electrical device, is subject to failure. In many applications, it is crucial that, if a failure occurs, the data be restored to the consistent state that existed prior to the failure. Consider a program to transfer ₦50 from account *A* to account *B*. If a system failure occurs during the execution of the program, it is possible that the ₦50 was removed from account *A* but was not credited to account *B*, resulting in an inconsistent database state. Clearly, it is essential to database consistency that either both the credit and debit occur, or that neither occur. That is, the funds transfer must be *atomic*—it must happen in its entirety or not at all. It is difficult to ensure atomicity in a conventional file-processing system.
- vi. **Concurrent-access anomalies.** For the sake of overall performance of the system and faster response, many systems allow multiple users to update the data simultaneously. In such an environment, interaction of concurrent updates may result in inconsistent data. Consider bank account *A*, containing ₦ 500. If two customers withdraw funds (say ₦50 and ₦100 respectively) from account *A* at about the same time, the result of the concurrent executions may leave the account in an incorrect (or inconsistent) state. Suppose that the programs executing on behalf of each withdrawal read the old balance, reduce that value by the amount being withdrawn, and write the result back. If the two programs run concurrently, they may both read the value ₦500, and write back ₦450 and ₦400, respectively. Depending on which one writes the value last, the account may contain ₦450 or ₦400, rather than the correct value of ₦350. To guard against this possibility, the system must maintain some form of supervision. But supervision is difficult to provide because data may be accessed by many different application programs that have not been coordinated previously.
- vii. **Security problems.** Not every user of the database system should be able to access all the data. For example, in a banking system, payroll personnel need to see only that part of the database that has information about the various bank employees. They do not need access to information about customer accounts. But, since application programs are added to the system in an ad hoc manner, enforcing such security constraints is difficult. These difficulties, among others, prompted the development of database systems. In what follows, we shall see the concepts and algorithms that enable database systems to solve the problems with file-processing systems. In most of this book,

we use a bank enterprise as a running example of a typical data-processing application found in a corporation.

DATABASE MANAGEMENT SYSTEM

A DBMS is a set of software programs that controls the system organisation, storage, management, and retrieval of data in database. DBMSs are categorized according to their data structures or types. The DBMS accepts requests for data from an application program and instructs the operating system to transfer the appropriate data. The queries and responses must be submitted and received according to a format that conforms to one or more protocols. When a DBMS is used, information systems can be changed more easily as the organisation's information requirements change. New categories of data can be added to the database without disruption to the existing system. The DBMS forms an interface between the user and the data in the base. A user does not directly access a database but does so through the DBMS. When a user requests for data, it is responsibility of the DBMS to retrieve the data from the base and present it to the user in the format the user wants. Database servers are dedicated computers that hold the actual databases and run only the DBMS and related software. DBMS are found at the heart of most database applications. DBMS may be built around a custom multitasking kernel with built-in networking support, but modern DBMSs typically rely on a standard operating system to provide these functions.

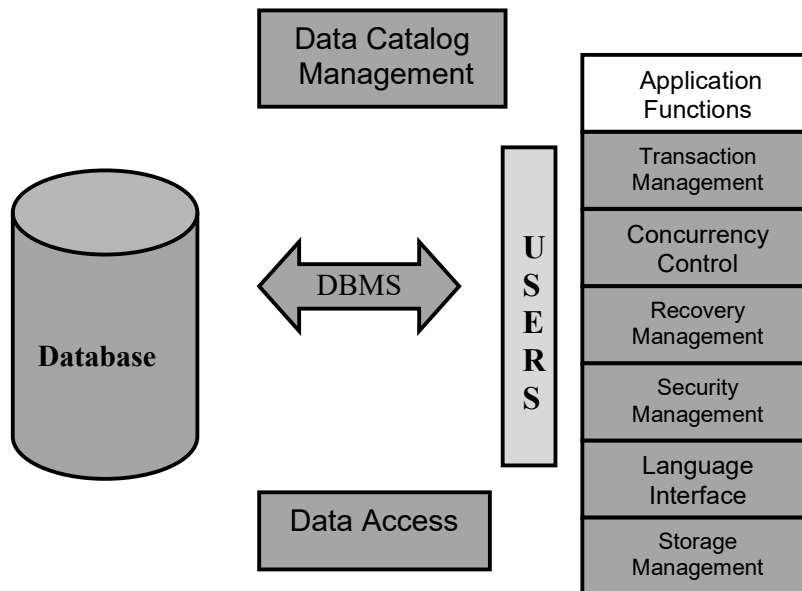


Figure 2.1: Components of a DBMS

FUNCTIONS OF A DATABASE MANAGEMENT SYSTEM

The Database Management System (DBMS) is a complex software program that is responsible for handling all access to database. The following are the function performed by a typical DBMS.

i. Transaction Management

A transaction is a sequence of database operations that represents a logical unit of work and that accesses a database and transforms it from one state to another. A transaction can update a record, delete (or) modify a set of records etc.

ii. Concurrency control

Concurrency control is the database management activity of coordinating the actions of database manipulating process that separate concurrently that access shared data and can potentially interfere with one another.

iii. Recovery Management

The recovery management system in a database ensures that the aborted or failed transactions create non adverse effect on the database or the other transitions.

iv. Security Management

Security refers to the protection of data against unauthorized access. Security mechanism of a DBMS makes sure that only authorized users are given access to the data in the database

v. Language Interface

The DBMS provides support languages used for the definition and manipulation of data in the database. The data structures are created using the data definition language commands. The data manipulation is done using the data manipulation commands.

vi. Storage Management

The DBMS provides a mechanism for management of permanent storage of the data. The internal schema defines how the data should be stored by the storage management mechanism and this storage manager interface with the operating system to access the physical storage.

vii. Data Catalog Management

Data catalog or Data Dictionary is a system database that contains descriptions of the data in the database (metadata). It contains information about data, relationships, constraints and the entire schema that organize these features in to a unified database.

TYPES OF DATABASE MANAGEMENT SYSTEM

A modelling language is a data modelling language to define the schema of each database hosted in the DBMS, according to database model. Database management system can be categorised according to the way the database they manage are organised. The four most common structures are:

a. Hierarchical Structure

Hierarchical structure was used in early mainframe DBMS. Records' relationships form a tree like model. The structure is simple but nonflexible because the relationship is confined to a one-to-many relationship. IBM's IMS system and RDM mobile are examples of a hierarchical data base system with multiple hierarchies over same data. The hierarchical structure is used primarily today for geographical information file systems.

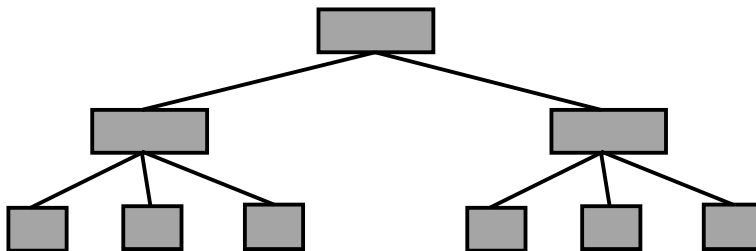


Figure 2.3: Hierarchical Database.

b. Network Structure

The network structure consists of more complex relationships. It can relate to many records and access them by following one of several paths. The structure allows for many-to-many relationship. It is commonly used in IDS and IDMS.

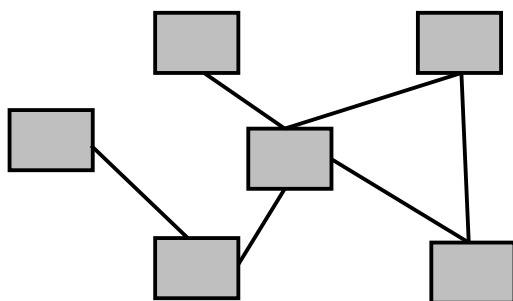


Figure 2.4: Network Structure

c. Relational Structure

The relational structure is the most commonly used today. It is used by mainframe, midrange and microcomputer systems. It uses two-dimensional rows and column to store data. The tables of records can be connected by common key values. It is used in numerous systems including IBM's DB2, Informix, Oracle, Sybase, Ms-Access, Foxbase, Paradox, Tandem and teradata. The model is not easy for the end user to run queries with because it may require a complex combination of many tables.

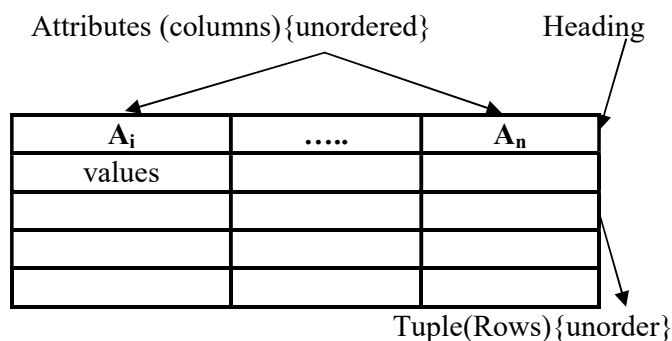


Figure 2.5: Relational Structure

d. Object-Oriented Structure

The Object-Oriented structure has the ability to handle graphics, pictures, voice and text, types of data, without difficulty unlike the other database structure. The structure is popular for multi-media web based applications. It was designed to work with object oriented programming languages such as java, C++, C# etc. the object-oriented structure is most widely used in ObjectStore and Versant.

CHARACTERISTICS OF DATA IN A DATABASE

The data in a database should have the following features:

- Shared:** Data in a database are shared among different users and applications.
- Persistence:** Data in a database exist permanently in the sense the data can live beyond the scope of the process that created it.
- Validity / Integrity / Correctness:** Data should be correct with Respect to the real world entity that they represent.
- Security:** Data should be protected from unauthorized access.
- Consistency:** Whenever more than are database represents related real – world values, the values should be consistent with respect to the relationship.
- Non redundancy:** No two data items in a database should represent the same real world entity.

...*Fundamentals of Database System Concepts*

- vii. **Independence:** The three levels in the schema (internal, conceptual and external) should be independent of each other so that the changes in the schema at one level should not affect the other levels.

BENEFITS OF DBMS

- i. Improved strategic use of corporate data
- ii. Reduced complexity of the organization's information systems environment
- iii. Reduced data redundancy and inconsistency
- iv. Enhanced data integrity
- v. Application-data independence
- vi. Improved security
- vii. Reduced application development and maintenance costs
- viii. Improved flexibility of information systems
- ix. Increased access and availability of data and information
- x. Logical & Physical data independence
- xi. Concurrent access anomalies.
- xii. Facilitate atomicity problem.
- xiii. Provides central control on the system through DBA.

LEVELS OF ABSTRACTION IN A DBMS

A database system is a collection of interrelated files and a set of programs that allow users to access and modify these files. A major purpose of a database system is to provide users with an *abstract* view of the data. That is, the system hides certain details of how the data are stored and maintained. For the system to be usable, it must retrieve data efficiently. The need for efficiency has led designers to use complex data structures to represent data in the database. Since many database-systems users are not computer trained, developers hide the complexity from users through several levels of abstraction, to simplify users' interactions with the system:

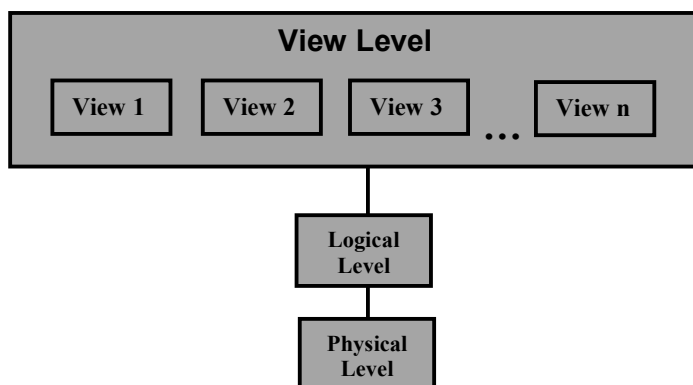


Figure 2.2: The three levels of data abstraction.

Physical level: The lowest level of abstraction describes *how* the data are actually stored. The physical level describes complex low-level data structures in detail.

Logical level: The next-higher level of abstraction describes *what* data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms

...Fundamentals of Database System Concepts

of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

View level. The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database.

INSTANCES AND SCHEMAS

The collection of information stored in the database at a particular moment is called an **instance** of the database. The overall design of the database is called the database **schema**. Schemas are changed infrequently, if at all. The concept of database schemas and instances can be understood by analogy to a program written in a programming language. A database schema corresponds to the variable declarations (along with associated type definitions) in a program. Each variable has a particular value at a given instant. The values of the variables in a program at a point in time correspond to an *instance* of a database schema.

Database systems have several schemas, partitioned according to the levels of abstraction. The **physical schema** describes the database design at the physical level, while the **logical schema** describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called **sub-schemas** that describe different views of the database.

Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit **physical data independence** if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes.

DATA MODELS

Underlying the structure of a database is the **data model**: a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. The entity-relationship model and the relational model both provide a way to describe the design of a database at the logical level.

The Entity-Relationship Model

The entity-relationship (E-R) data model is based on a perception of a real world that consists of a collection of basic objects, called *entities*, and of *relationships* among these objects. An entity is a “thing” or “object” in the real world that is distinguishable from other objects. For example, each person is an entity, and bank accounts can be considered as entities. Entities are described in a database by a set of attributes. For example, the attributes *account-number* and *balance* may describe one particular account in a bank, and they form attributes of the *account* entity set. Similarly, attributes *customer-name*, *customer-street* address and *customer-city* may describe a *customer* entity.

An extra attribute *customer-id* is used to uniquely identify customers (since it may be possible to have two customers with the same name, street address, and city). A unique customer identifier must be assigned to each customer.

A **relationship** is an association among several entities. For example, a *depositor* relationship associates a customer with each account that she has. The set of all entities of the same type and the set

...*Fundamentals of Database System Concepts*

of all relationships of the same type are termed an **entity set** and **relationship set**, respectively. The overall logical structure (schema) of a database can be expressed graphically by an *E-R diagram*. Details of E-R diagram to be discussed in later chapter.

The Relational Model

The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. Figure 2.3 below presents a sample relational database comprising three tables: One show details of bank customers, the second shows accounts, and the third shows which accounts belong to which customers.

<i>Customer-Id</i>	<i>Customer-name</i>	<i>Customer-street</i>	<i>Customer-city</i>
001	Shola	2, Muritala Rd.	Ilorin
002	Abdullahi	10, Shango St.	Kano
003	Okechukwu	4, Chinedu St.	Abia.
004	Omosetan	3, Gbodija Rd.	Ibadan.

(a) The customer table

<i>Account-number</i>	<i>Balance</i>
500270-1234	1000
500270-1235	500
500270-1236	700
500270-1237	650

(b) The account table

<i>Customer-Id</i>	<i>Account-number</i>
001	500270-1234
002	500270-1235
003	500270-1236
004	500270-1237

(c) The depositor table

Figure 2.3: A sample relational database.

The relational model is an example of a record-based model. Record-based models are so named because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each record type defines a fixed number of fields, or attributes. The columns of the table correspond to the attributes of the record type. The relational model hides such low-level implementation details from database developers and users. The relational data model is the most widely used data model, and a vast majority of current database systems are based on the relational model. Later chapter covers the relational model in detail. The relational model is at a lower level of abstraction than the E-R model. Database designs are often carried out in the E-R model, and then translated to the relational model.

DATABASE LANGUAGES

A database system provides a **data definition language** to specify the database schema and a **data manipulation language** to express database queries and updates. In practice, the data definition and data manipulation languages are not two separate languages; instead they simply form parts of a single database language, such as the widely used SQL language.

Data-Definition Language

...Fundamentals of Database System Concepts

A database schema is specified by a set of definitions expressed by a special language called a **data-definition language (DDL)**. For instance, the following statement in the SQL language defines the *account* table:

```
create table account
(account-number char(10),
balance integer)
```

Execution of the above DDL statement creates the *account* table. In addition, it updates a special set of tables called the **data dictionary** or **data directory**. A data dictionary contains **metadata**—that is, data about data. The schema of a table is an example of metadata. A database system consults the data dictionary before reading or modifying actual data.

We specify the storage structure and access methods used by the database system by a set of statements in a special type of DDL called a **data storage and definition** language. These statements define the implementation details of the database schemas, which are usually hidden from the users. The data values stored in the database must satisfy certain **consistency constraints**. For example, suppose the balance on an account should not fall below ₦100. The DDL provides facilities to specify such constraints. The database systems check these constraints every time the database is updated.

Data-Manipulation Language

Data manipulation is

- The retrieval of information stored in the database
- The insertion of new information into the database
- The deletion of information from the database
- The modification of information stored in the database

A data-manipulation language (DML) is a language that enables users to access or manipulate data as organized by the appropriate data model. There are basically two types:

- **Procedural DMLs** require a user to specify *what* data are needed and *how* to get those data.
- **Declarative DMLs** (also referred to as **nonprocedural DMLs**) require a user to specify *what* data are needed *without* specifying how to get those data.

Declarative DMLs are usually easier to learn and use than are procedural DMLs. However, since a user does not have to specify how to get the data, the database system has to figure out an efficient means of accessing data. The DML component of the SQL language is nonprocedural.

A **query** is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a **query language**. Although technically incorrect, it is common practice to use the terms *query language* and *data manipulation language* synonymously.

DATABASE ACCESS FROM APPLICATION PROGRAMS

Application programs are programs that are used to interact with the database. Application programs are usually written in a *host* language, such as COBOL, C, C++, or Java. Examples in a banking system are programs that generate payroll checks, debit accounts, credit accounts, or transfer funds between accounts. To access the database, DML statements need to be executed from the host language. There are two ways to do this:

- By providing an application program interface (set of procedures) that can be used to send DML and DDL statements to the database, and retrieve the results. The Open Database Connectivity (ODBC) standard defined by Microsoft for use with the C language is a commonly used application program

...Fundamentals of Database System Concepts

interface standard. The Java Database Connectivity (JDBC) standard provides corresponding features to the Java language.

- By extending the host language syntax to embed DML calls within the host language program. Usually, a special character prefaces DML calls, and a pre-processor, called the **DML pre-compiler**, converts the DML statements to normal procedure calls in the host language.

DATABASE USERS AND ADMINISTRATORS

A primary goal of a database system is to retrieve information from and store new information in the database. People who work with a database can be categorized as database users or database administrators.

DATABASE USERS AND USER INTERFACES

There are four different types of database-system users, differentiated by the way they expect to interact with the system. Different types of user interfaces have been designed for the different types of users.

- **Naive users** are unsophisticated users who interact with the system by invoking one of the application programs that have been written previously. For example, a bank teller who needs to transfer \$50 from account *A* to account *B* invokes a program called *transfer*. This program asks the teller for the amount of money to be transferred, the account from which the money is to be transferred, and the account to which the money is to be transferred. The typical user interface for naive users is a forms interface, where the user can fill in appropriate fields of the form. Naive users may also simply read *reports* generated from the database.
- **Application programmers** are computer professionals who write application programs. Application programmers can choose from many tools to develop user interfaces. **Rapid application development (RAD)** tools are tools that enable an application programmer to construct forms and reports without writing a program. There are also special types of programming languages that combine imperative control structures (for example, for loops, while loops and if-then-else statements) with statements of the data manipulation language. These languages, sometimes called *fourth-generation languages*, often include special features to facilitate the generation of forms and the display of data on the screen. Most major commercial database systems include a fourth generation language.
- **Sophisticated users** interact with the system without writing programs. Instead, they form their requests in a database query language. They submit each such query to a **query processor**, whose function is to break down DML statements into instructions that the storage manager understands. Analysts who submit queries to explore data in the database fall in this category.
- **Specialized users** are sophisticated users who write specialized database applications that do not fit into the traditional data-processing framework. Among these applications are computer-aided design systems, knowledge base and expert systems, systems that store data with complex data types (for example, graphics data and audio data), and environment-modelling systems.

DATABASE ADMINISTRATOR

One of the main reasons for using DBMSs is to have central control of both the data and the programs that access those data. A person who has such central control over the system is called a **database administrator (DBA)**. The functions of a DBA include:

- i. **Schema definition.** The DBA creates the original database schema by executing a set of data definition statements in the DDL.
- ii. **Storage structure and access-method definition.**

...Fundamentals of Database System Concepts

- iii. **Schema and physical-organization modification.** The DBA carries out changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve performance.
- iv. **Granting of authorization for data access.** By granting different types of authorization, the database administrator can regulate which parts of the database various users can access. The authorization information is kept in a special system structure that the database system consults whenever someone attempts to access the data in the system.
- v. **Routine maintenance.** Examples of the database administrator's routine maintenance activities are:
 - Periodically backing up the database, either onto tapes or onto remote servers, to prevent loss of data in case of disasters such as flooding.
 - Ensuring that enough free disk space is available for normal operations, and upgrading disk space as required.
 - Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users

EXERCISE 2

1. Why would you choose a database system instead of simply storing data in operating system files?
2. Explain the difference between logical and physical data independence.
3. What are the main functions of database administrator?
4. List 5 programming language that are procedural and two that are non-procedural. Which group is easier to learn and use? Explain your answer.
5. Consider a two-dimensional integer array of size $n \times m$ that is to be used in your favourite programming language. Using the array as an example, illustrate the difference
 - (a) between the three levels of data abstraction, and
 - (b) between a schema and instances.

CHAPTER THREE

ENTITY - RELATIONSHIP MODEL

INTRODUCTION

A data model is a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. The entity–relationship (E-R) model is a high-level data model. It is based on a perception of a real world that consists of a collection of basic objects, called *entities*, and of *relationships* among these objects. The relational model is a lower-level model. It uses a collection of tables to represent both data and the relationships among those data. The entity-relationship (ER) data model allows us to describe the data involved in a real world enterprise in terms of objects and their relationships. The entity-relationship (E-R) data model perceives the real world as consisting of basic objects, called *entities*, and *relationships* among these objects. It was developed to facilitate database design by allowing specification of an *enterprise schema*, which represents the overall logical structure of a database. The E-R data model is one of several semantic data models; the semantic aspect of the model lies in its representation of the meaning of the data. The E-R model is very useful in mapping the meanings and interactions of real-world enterprises onto a conceptual schema. Because of this usefulness, many database-design tools draw on concepts from the E-R model.

BASIC CONCEPTS

The E-R data model employs three basic notions: entity sets, relationship sets, and attributes.

Entity Sets

An entity is a “thing” or “object” in the real world that is distinguishable from all other objects. For example, each person in an enterprise is an entity. An entity has a set of properties, and the values for some set of properties may uniquely identify an entity. For instance, a person may have a *person-id* property whose value uniquely identifies that person. Thus, the value 001 for *person-id* would uniquely identify one particular person in an organization. An entity may be *concrete*, such as a person or a book, or it may be *abstract*, such as a loan, or a holiday, or a concept.

An *entity set* is a set of entities of the same type that share the same properties, or attributes. The set of all persons who are customers at a given bank, for example, can be defined as the entity set *customer*. Similarly, the entity set *loan* might represent the set of all loans awarded by a particular bank. The individual entities that constitute a set are said to be the *extension* of the entity set. Thus, all the individual bank customers are the extension of the entity set *customer*. Entity sets do not need to be disjoint. For example, it is possible to define the entity set of all employees of a bank (*employee*) and the entity set of all customers of the bank (*customer*). A *person* entity may be an *employee* entity, a *customer* entity, both, or neither. An entity is represented by a set of **attributes**.

Attributes

Attributes are descriptive properties possessed by each member of an entity set. The designation of an attribute for an entity set expresses that the database stores similar information concerning each entity in the entity set; however, each entity may have its own value for each attribute. Possible attributes of the *customer* entity set are *customer-id*, *customer-name*, *customer-street*, and *customer-city*. In real life, there would be further attributes, such as street number, apartment number, state, postal code, and country.

Each entity has a **value** for each of its attributes. For instance, a particular *customer* entity may have the value 001 for *customer-id*, the value Ayodele for *customer-name*, the value Taiwo Rd for *customer-street*, and the value Ilorin for *customer-city*. The *customer-id* attribute is used to uniquely identify customers, since there may be more than one customer with the same name, street, and city. In general the enterprise would have to create and assign a unique identifier for each customer. For each

...*Fundamentals of Database System Concepts*

attribute, there is a set of permitted values, called the **domain**, or **value set** (data type), of that attribute. The domain of attribute *customer-name* might be the set of all text strings of a certain length.

A database thus includes a collection of entity sets, each of which contains any number of entities of the same type. Figure below shows part of a bank database that consists of two entity sets: *customer* and *loan*.

<i>Customer-Id</i>	<i>Customer-name</i>	<i>Customer-street</i>	<i>Customer-city</i>
001	Shola	Muritala Rd.	Ilorin
002	Abdullahi	Shango St.	Kano
003	Okechukwu	Chinedu St.	Abia.
004	Omosetan	Gbodija Rd.	Ibadan.

(a) *The customer table*

<i>Loan type</i>	<i>Amount</i>
L-5	2000
L-2	1500
L-10	4000
L-8	2500

(b) *Loan table*

Figure 3.1: Entity set for customer and loan

Formally, an attribute of an entity set is a function that maps from the entity set into a domain. Since an entity set may have several attributes, each entity can be described by a set of (attribute, data value) pairs, one pair for each attribute of the entity set. For example, a particular *customer* entity may be described by the set $\{(customer-id, 001), (customer-name, Shola), (customer-street, Muritala), (customer-city, Ilorin)\}$, meaning that the entity describes a person named Shola whose customer identifier is 001 and who resides at Murital road in Ilorin.

Types of attributes

An attribute, as used in the E-R model, can be characterized by the following attribute types.

- i. **Simple and composite attributes:** Simple attributes are the attributes that are not divided into subparts. Composite attributes, on the other hand, can be divided into subparts (that is, other attributes). For example, an attribute *name* could be structured as a composite attribute consisting of *first-name*, *middle-initial*, and *last-name*. Using composite attributes in a design schema is a good choice if a user will wish to refer to an entire attribute on some occasions and to only a component of the attribute on other occasions. Suppose we were to substitute for the *customer* entity-set attributes *customer-street* and *customer-city* the composite attribute *address* with the attributes; *street*, *city*, *state*, and *zip-code*. Composite attributes help us to group together related attributes, making the modeling cleaner. In the composite attribute *address*, its component attribute *street* can be further divided into *street-number*, *street-name*, and *apartment-number*.

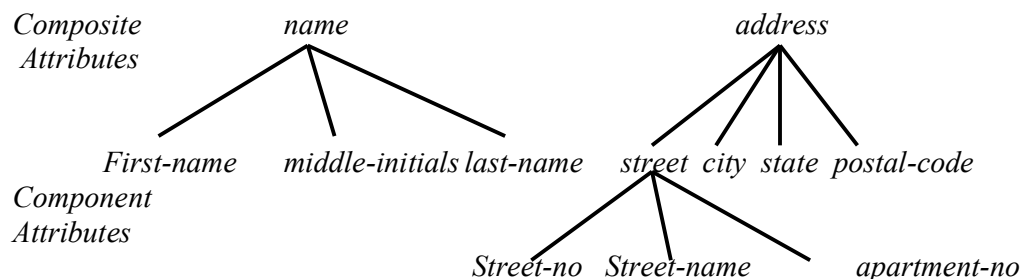


Figure 3.2: Composite attributes

- ii. **Single-valued and multi-valued attributes:** The attributes in the examples all have a single value for a particular entity. For instance, the *loan-number* attribute for a specific loan entity refers to only one loan number. Such attributes are said to be **single valued**. There may be instances where an attribute has a set of values for a specific entity. Consider an *employee* entity set with the attribute *phone-number*. An employee may have zero, one, or several phone numbers, and different employees may have different numbers of phones. This type of attribute is said to be **multi-valued**. For example, an attribute *dependent-name* of the *employee* entity set would be multi-valued, since any particular employee may have zero, one, or more dependent(s). However, upper and lower bounds may be placed on the number of values in a multi-valued attribute. For example, a bank may limit the number of phone numbers recorded for a single customer to two. Placing bounds in this case expresses that the *phone-number* attribute of the *customer* entity set may have between zero and two values.
- iii **Derived attribute:** The value for this type of attribute can be derived from the values of other related attributes or entities. For example, suppose that the *customer* entity set has an attribute *age*, which indicates the customer's age. If the *customer* entity set also has an attribute *date-of-birth*, we can calculate *age* from *date-of-birth* and the current date. Thus, *age* is a derived attribute. In this case, *date-of-birth* may be referred to as a *base* attribute, or a *stored* attribute. The value of a derived attribute is not stored, but is computed when required.

An attribute takes a **null** value when an entity does not have a value for it. The *null* value may indicate “not applicable”—that is, that the value does not exist for the entity. For example, one may have no middle name. *Null* can also designate that an attribute value is *unknown*. An unknown value may be either *missing* (the value does exist, but we do not have that information) or *not known* (we do not know whether or not the value actually exists). For instance, if the *name* value for a particular customer is *null*, we assume that the value is missing, since every customer must have a name. A null value for the *apartment-number* attribute could mean that the address does not include an apartment number (not applicable), that an apartment number exists but we do not know what it is (missing), or that we do not know whether or not an apartment number is part of the customer's address (unknown).

A database for a banking enterprise may include a number of different entity sets. For example, in addition to keeping track of customers and loans, the bank also provides accounts, which are represented by the entity set *account* with attributes *account-number* and *balance*. Also, if the bank has a number of different branches, then we may keep information about all the branches of the bank. Each *branch* entity set may be described by the attributes *branch-name*, *branch-city*, and *assets*.

Relationship Sets

A relationship is an association among several entities. For example, we can define a relationship that associates customer Shola with loan L-5. This relationship specifies that Shola is a customer with loan number L-5.

A **relationship set** is a set of relationships of the same type. Consider the two entity sets *customer* and *loan* in Figure 3.1 above. We define the relationship set *borrower* to denote the association between customers and the bank loans that the customers have.

As another example, consider the two entity sets *loan* and *branch*. We can define the relationship set *loan-branch* to denote the association between a bank loan and the branch in which that loan is maintained. The association between entity sets is referred to as participation; that is, the entity sets **participate** in relationship set. A **relationship instance** in an E-R schema represents an association between the named entities in the real-world enterprise that is being modeled. For example, the individual *customer* entity Shola, who has customer identifier 001, and the *loan* entity L-5 participate in a relationship instance of *borrower*. This relationship instance represents that, in the real-world enterprise, the person called Shola who holds *customer-id* 001 has taken the loan that is numbered L-5.

The function that an entity plays in a relationship is called that entity's **role**. Since entity sets participating in a relationship set are generally distinct, roles are implicit and are not usually specified. However, they are useful when the meaning of a relationship needs clarification. Such is the case when the entity sets of a relationship set are not distinct; that is, the same entity set participates in a relationship set more than once, in different roles. In this type of relationship set, sometimes called a **recursive** relationship set, explicit role names are necessary to specify how an entity participates in a relationship instance. For example, consider an entity set *employee* that records information about all the employees of the bank. We may have a relationship set *works-for* that is modeled by ordered pairs of *employee* entities. The first employee of a pair takes the role of *worker*, whereas the second takes the role of *manager*. In this way, all relationships of *works-for* are characterized by (worker, manager) pairs; (manager, worker) pairs are excluded.

A relationship may also have attributes called **descriptive attributes**. Consider a relationship set *depositor* with entity sets *customer* and *account*. We could associate the attribute *access-date* to that relationship to specify the most recent date on which a customer accessed an account. The *depositor* relationship among the entities corresponding to customer Shola and account 500270-1234 has the value "23 January 2011" for attribute *access-date*, which means that the most recent date that Shola accessed account 500270-1234 was 23 January 2011.

However, there can be more than one relationship set involving the same entity sets. In the example the *customer* and *loan* entity sets participate in the relationship set *borrower*. Additionally, suppose each loan must have another customer who serves as a guarantor for the loan. Then the *customer* and *loan* entity sets may participate in another relationship set, *guarantor*. The relationship sets *borrower* and *loan-branch* provide an example of a **binary** relationship set—that is, one that involves two entity sets. Most of the relationship sets in a database system are binary. Occasionally, however, relationship sets involve more than two entity sets.

As an example, consider the entity sets *employee*, *branch*, and *job*. Examples of *job* entities could include manager, teller, auditor, and so on. Job entities may have the attributes *title* and *level*. The relationship set *works-on* among *employee*, *branch*, and *job* is an example of a ternary relationship. A ternary relationship among Ayodele, Offa, and manager indicates that Ayodele acts as a manager at the Offa branch. Ayodele could also act as auditor at the Ilorin branch, which would be represented by another relationship. Yet another relationship could be between Olufemi, Ilorin, and teller, indicating Olufemi acts as a teller at the Ilorin branch. The number of entity sets that participate in a relationship set is also the **degree** of the relationship set. A binary relationship set is of degree 2; a ternary relationship set is of degree 3.

CONSTRAINTS

An E-R enterprise schema may define certain constraints to which the contents of a database must conform. **Mapping Cardinalities** or **cardinality ratios**, express the number of entities to which another entity can be associated via a relationship set. Mapping cardinalities are most useful in describing binary relationship sets, although they can contribute to the description of relationship sets that involve more than two entity sets. For a binary relationship set R between entity sets A and B , the mapping cardinality must be one of the following:

- i. **One-to-one:** An entity in A is associated with *at most* one entity in B , and an entity in B is associated with *at most* one entity in A .
- ii. **One-to-many:** An entity in A is associated with any number (zero or more) of entities in B . An entity in B , however, can be associated with *at most* one entity in A .
- iii. **Many-to-one.** An entity in A is associated with *at most* one entity in B . An entity in B , however, can be associated with any number (zero or more) of entities in A .
- iv. **Many-to-many.** An entity in A is associated with any number (zero or more) of entities in B , and an entity in B is associated with any number (zero or more) of entities in A .

The appropriate mapping cardinality for a particular relationship set obviously depends on the real-world situation that the relationship set is modeling. As an illustration, consider the *borrower* relationship set. If, in a particular bank, a loan can belong to only one customer, and a customer can have several loans, then the relationship set from *customer* to *loan* is one to many. If a loan can belong to several customers (as can loans taken jointly by several business partners), the relationship set is many to many.

KEYS

There must be a way to specify how entities within a given entity set are distinguished. Conceptually, individual entities are distinct; from a database perspective, however, the difference among them must be expressed in terms of their attributes. Therefore, the values of the attribute values of an entity must be such that they can *uniquely identify* the entity. In other words, no two entities in an entity set are allowed to have exactly the same value for all attributes. A *key* allows us to identify a set of attributes that suffice to distinguish entities from each other. Keys also help uniquely identify relationships, and thus distinguish relationships from each other.

A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely an entity in the entity set. For example, the *customer-id* attribute of the entity set *customer* is sufficient to distinguish one *customer* entity from another. Thus, *customer-id* is a superkey. Similarly, the combination of *customer-name* and *customer-id* is a superkey for the entity set *customer*. The *customer-name* attribute of *customer* is not a superkey, because several people might have the same name. The concept of a superkey is not sufficient for our purposes, since, as we saw, a superkey may contain extraneous attributes. If K is a superkey, then so is any superset of K . We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called **candidate keys**.

It is possible that several distinct sets of attributes could serve as a candidate key. Suppose that a combination of *customer-name* and *customer-street* is sufficient to distinguish among members of the *customer* entity set. Then, both $\{customer-id\}$ and $\{customer-name, customer-street\}$ are candidate keys. Although the attributes *customer-id* and *customer-name* together can distinguish *customer* entities, their combination does not form a candidate key, since the attribute *customer-id* alone is a candidate key.

The term **primary key** is used to denote a candidate key that is chosen by the database designer as the principal means of identifying entities within an entity set. A key (primary, candidate, and super) is a property of the entity set, rather than of the individual entities. Any two individual entities in the set are prohibited from having the same value on the key attributes at the same time. The designation of a key represents a constraint in the real-world enterprise being modeled.

ENTITY-RELATIONSHIP DIAGRAM

E-R diagram can express the overall logical structure of a database graphically. E-R diagrams are simple and clear qualities that may well account in large part for the widespread use of the E-R model. Such a diagram consists of the following major components:

Below are some of the symbols used in the E-R notation:

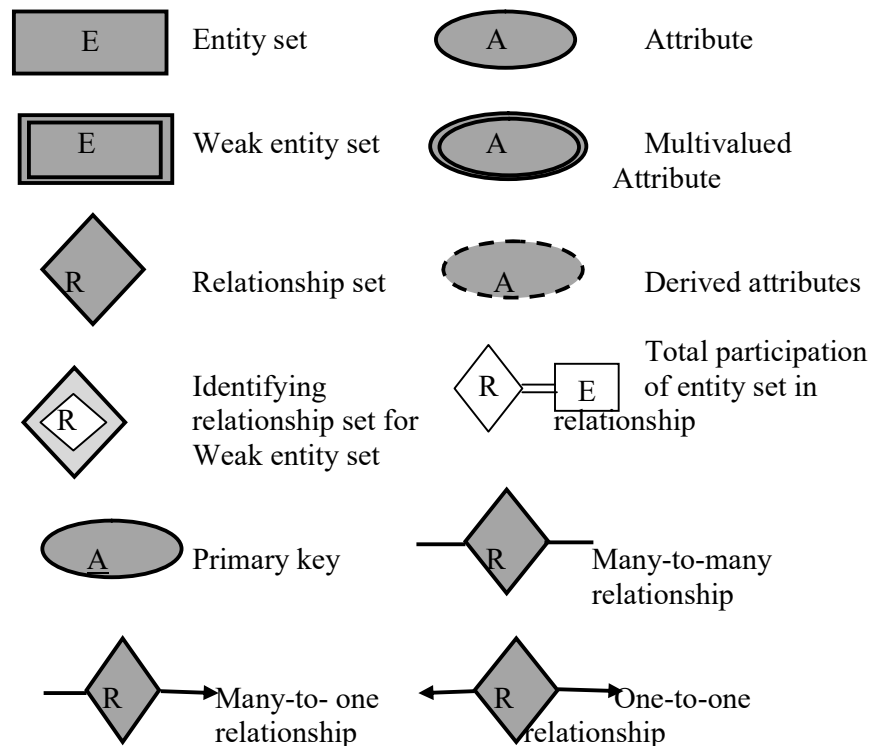


Figure 3.3: Symbols used for E-R notation

- i. **Rectangles**, which represent entity sets.
- ii. **Ellipses**, which represent attributes.
- iii. **Diamonds**, which represent relationship sets.
- iv. **Lines**, which link attributes to entity sets and entity sets to relationship sets.
- v. **Double ellipses**, which represent multi-valued attributes.
- vi. **Dashed ellipses**, which denote derived attributes.
- vii. **Double lines**, which indicate total participation of an entity in a relationship set.
- viii. **Double rectangles**, which represent weak entity sets.

Consider the entity-relationship diagram in Figure 3.4, which consists of two entity sets, *customer* and *loan*, related through a binary relationship set *borrower*. The attributes associated with *customer* are *customer-id*, *customer-name*, *customer-street*, and *customer-city*. The attributes associated with *loan* are *loan-number* and *amount*. In Figure 3.4, attributes of an entity set that are members of the primary

...Fundamentals of Database System Concepts

key are underlined. The relationship set *borrower* may be many-to-many, one-to-many, many-to-one, or one-to-one. To distinguish among these types, we draw either a directed line ($\rightarrow$) or an undirected line ($—$) between the relationship set and the entity set in question.

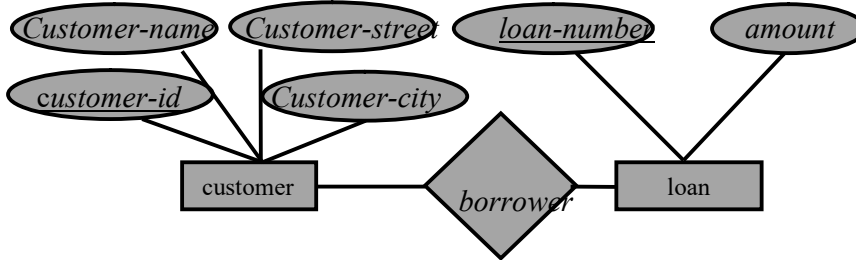
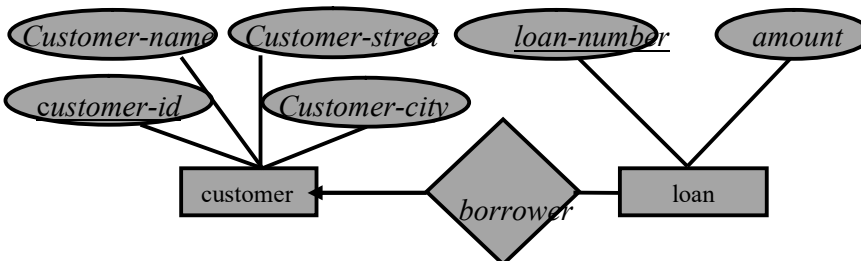


Figure 3.4: E-R diagram corresponding to customers and loans

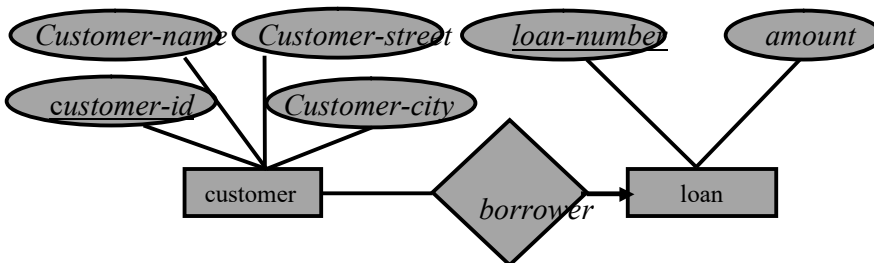
- A directed line from the relationship set *borrower* to the entity set *loan* specifies that *borrower* is either one-to-one or many-to-one relationship set, from *customer* to *loan*; *borrower* cannot be a many-to-many or a one-to-many relationship set from *customer* to *loan*.
- An undirected line from the relationship set *borrower* to the entity set *loan* specifies that *borrower* is either a many-to-many or one-to-many relationship set from *customer* to *loan*.

Returning to the E-R diagram of Figure 3.4 above, we see that the relationship set *borrower* is many-to-many. If the relationship set *borrower* were one-to-many, from *customer* to *loan*, then the line from *borrower* to *customer* would be directed, with an arrow pointing to the *customer* entity set (Figure 3.5a). Similarly,



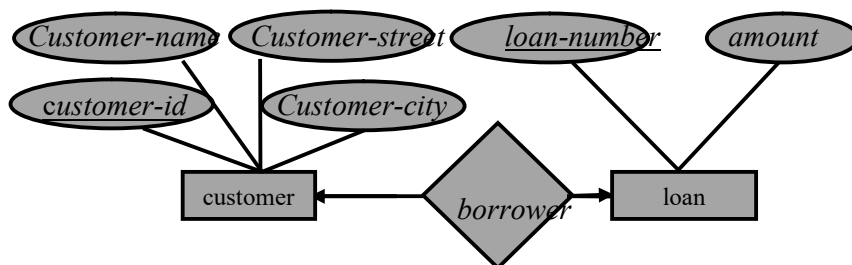
(a) One-to-many relationship

Then if the relationship set *borrower* were many-to-one from *customer* to *loan*, then the line from *borrower* to *loan* would have an arrow pointing to the *loan* entity set (Figure 3.5b).



(b) Many-to-one relationship

Finally, if the relationship set *borrower* were one-to-one, then both lines from *borrower* would have arrows: one pointing to the *loan* entity set and one pointing to the *customer* entity set (Figure 3.5c).



(b) One-to-one relationship

Figure 3.5: Different relationships

If a relationship set has also some attributes associated with it, then we link these attributes to that relationship set. For example, in Figure 3.6 below, we have the *access-date* descriptive attribute attached to the relationship set *depositor* to specify the most recent date on which a customer accessed that account.

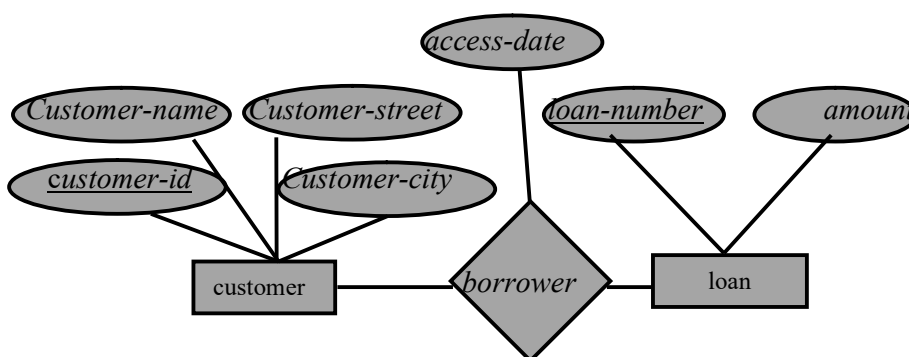


Figure 3.6: E-R diagram with an attribute attached to a relationship set

However, figure 3.7 below shows how composite attributes can be represented in the E-R notation. Here, a composite attribute *name*, with component attributes *first-name*, *middle-initial*, and *last-name*, replaces the simple attribute *customer-name* of *customer*.

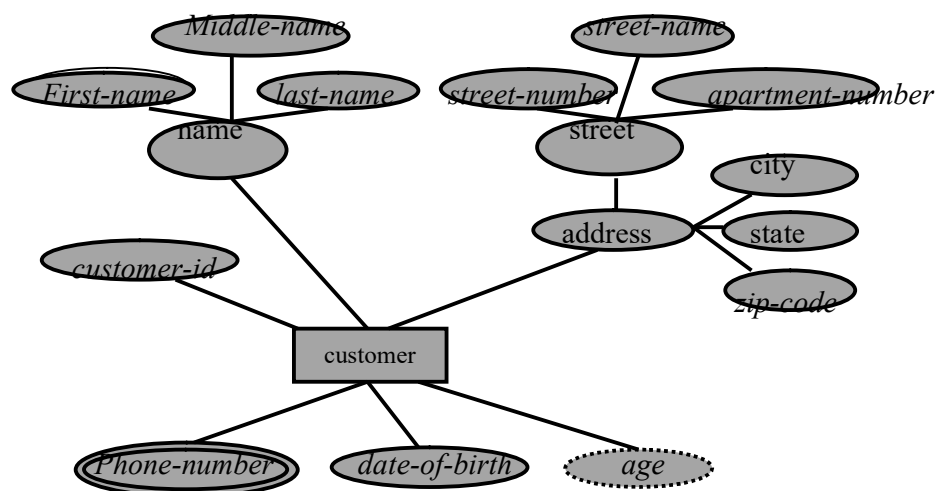


Figure 3.7: E-R diagram with composite, multivalued and derived attributes

Also, a composite attribute *address*, whose component attributes are *street*, *city*, *state*, and *zip-code* replaces the attributes *customer-street* and *customer-city* of *customer*. The attribute *street* is itself a

...Fundamentals of Database System Concepts

composite attribute whose component attributes are *street-number*, *street-name*, and *apartment number*. The figure also illustrates a multivalued attribute *phone-number*, depicted by a double ellipse, and a derived attribute *age*, depicted by a dashed ellipse.

The figure below indicates roles in E-R diagrams by labeling the lines that connect diamonds to rectangles. It shows the role indicators *manager* and *worker* between the *employee* entity set and the *works-for* relationship set.

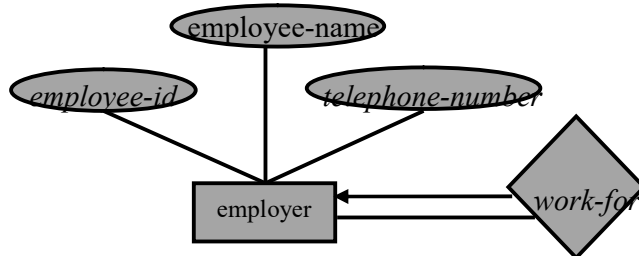


Figure 3.8: E-R diagram with role indicators

Non-binary relationship sets can be specified easily in an E-R diagram. The figure 3.9 below consists of the three entity sets *employee*, *job*, and *branch*, related through the relationship set *works-on*. We can specify some types of many-to-one relationships in the case of non-binary relationship sets. Suppose an employee can have at most one job in each branch (for example, Shola cannot be a manager and an auditor at the same branch). This constraint can be specified by an arrow pointing to *job* on the edge from *works-on*. We permit at most one arrow out of a relationship set, since an E-R diagram with two or more arrows out of a non-binary relationship set can be interpreted in two ways

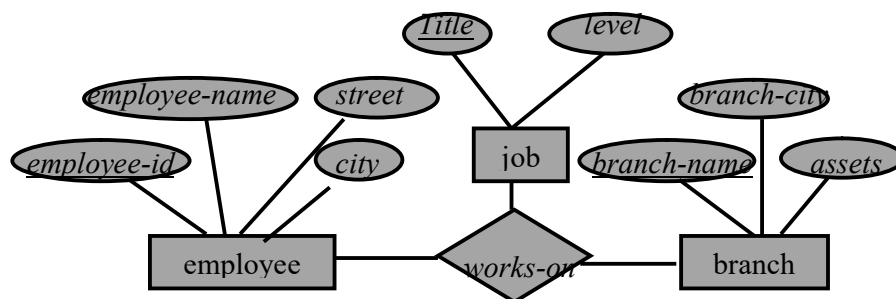


Figure 3.9: E-R diagram with a ternary relationship

Double lines are used in an E-R diagram to indicate that the participation of an entity set in a relationship set is total; that is, each entity in the entity set occurs in at least one relationship in that relationship set.

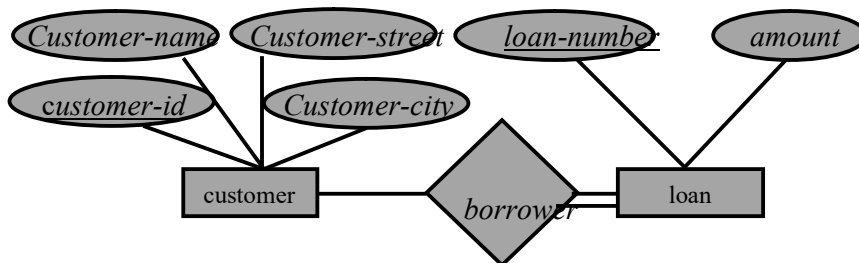


Figure 3.10: E-R diagram with total participation of an entity in a relationship set

For instance, consider the relationship *borrower* between customers and loans. A double line from *loan* to *borrower*, indicates that each loan must have at least one associated customer.

WEAK ENTITY SETS

An entity set may not have sufficient attributes to form a primary key. Such an entity set is termed a **weak entity set**. An entity set that has a primary key is termed a **strong entity set**. As an illustration, consider the entity set *payment*, which has the three attributes: *payment-number*, *payment-date*, and *payment amount*. Payment numbers are typically sequential numbers, starting from 1, generated separately for each loan. Thus, although each *payment* entity is distinct, payments for different loans may share the same payment number. Thus, this entity set does not have a primary key; it is a weak entity set. For a weak entity set to be meaningful, it must be associated with another entity set, called the **identifying** or **owner entity set**. Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set. The identifying entity set is said to **own** the weak entity set that it identifies. The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**. The identifying relationship is many to one from the weak entity set to the identifying entity set, and the participation of the weak entity set in the relationship is total. In the example, the identifying entity set for *payment* is *loan*, and a relationship *loan-payment* that associates *payment* entities with their corresponding *loan* entities is the identifying relationship.

Although a weak entity set does not have a primary key, we nevertheless need a means of distinguishing among all those entities in the weak entity set that depend on one particular strong entity. The **discriminator** of a weak entity set is a set of attributes that allows this distinction to be made. For example, the discriminator of the weak entity set *payment* is the attribute *payment-number*, since, for each loan, a payment number uniquely identifies one single payment for that loan. The discriminator of a weak entity set is also called the *partial key* of the entity set. The primary key of a weak entity set is formed by the primary key of the identifying entity set, plus the weak entity set's discriminator. In the case of the entity set *payment*, its primary key is $\{loan-number, payment-number\}$, where *loan-number* is the primary key of the identifying entity set, namely *loan*, and *payment-number* distinguishes *payment* entities within the same loan.

The identifying relationship set should have no descriptive attributes, since any required attributes can be associated with the weak entity set. A weak entity set can participate in relationships other than the identifying relationship. For instance, the *payment* entity could participate in a relationship with the *account* entity set, identifying the account from which the payment was made. A weak entity set may participate as owner in an identifying relationship with another weak entity set. It is also possible to have a weak entity set with more than one identifying entity set. A particular weak entity would then be identified by a combination of entities, one from each identifying entity set. The primary key of the weak entity set would consist of the union of the primary keys of the identifying entity sets, plus the discriminator of the weak entity set. In E-R diagrams, a doubly outlined box indicates a weak entity set, and a doubly outlined diamond indicates the corresponding identifying relationship. In Figure 3.11 below, the weak entity set *payment* depends on the strong entity set *loan* via the relationship set *loan-payment*.

The figure also illustrates the use of double lines to indicate *total participation*—the participation of the (weak) entity set *payment* in the relationship *loan-payment* is total, meaning that every payment must be related via *loan-payment* to some loan. Finally, the arrow from *loan-payment* to *loan* indicates that each payment is for a single loan. The discriminator of a weak entity set also is underlined, but with a dashed, rather than a solid, line. In some cases, the database designer may choose to express a weak entity set as a multi-valued composite attribute of the owner entity set. In our example, this alternative would require that the entity set *loan* have a multi-valued, composite attribute *payment*, consisting of *payment-number*, *payment-date*, and *payment-amount*. A weak entity set may be more

...Fundamentals of Database System Concepts

appropriately modeled as an attribute if it participates in only the identifying relationship, and if it has few attributes. Conversely, a weak-entity set representation will more aptly model a situation where the set participates in relationships other than the identifying relationship, and where the weak entity set has several attributes.

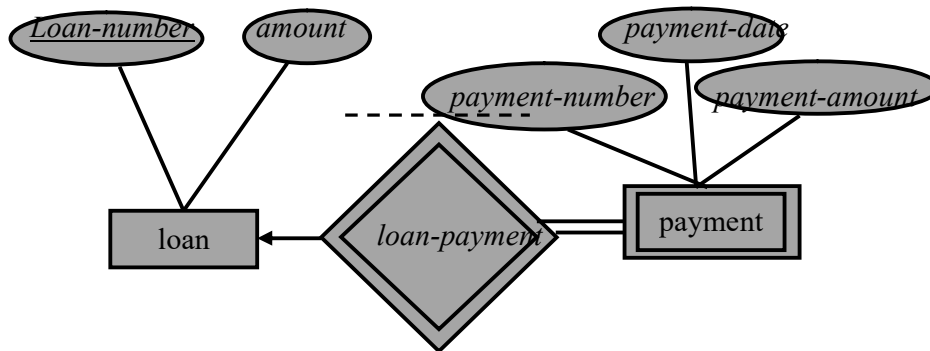


Figure 3.11: E-R diagram with a weak entity set.

EXERCISE 3

1. Explain the distinctions among the terms primary key, candidate key and superkey.
2. Discuss the types of attributes know to you with examples.
3. Consider the following information about a university database:
 - Professors have an SSN, a name, an age, a rank, and a research specialty.
 - Projects have a project number, a sponsor name (e.g., NSF), a starting date, an ending date, and a budget.
 - Graduate students have an SSN, a name, an age, and a degree program (e.g., M.S. or Ph.D.).
 - Each project is managed by one professor (known as the project's principal investigator).
 - Each project is worked on by one or more professors (known as the project's co-investigators).
 - Professors can manage and/or work on multiple projects.
 - Each project is worked on by one or more graduate students (known as the project's research assistants).
 - When graduate students work on a project, a professor must supervise their work on the project. Graduate students can work on multiple projects, in which case they will have a (potentially different) supervisor for each one.
 - Departments have a department number, a department name, and a main office.
 - Departments have a professor (known as the chairman) who runs the department.
 - Professors work in one or more departments and for each department that they work in, a time percentage is associated with their job.
 - Graduate students have one major department in which they are working on their degree.
 - Each graduate student has another, more senior graduate student (known as a student advisor) who advises him or her on what courses to take.

Design and draw an ER diagram that captures the information about the university. Use only the basic ER model here, that is, entities, relationships, and attributes. Be sure to indicate any key and participation constraints.

4. A company database needs to store information about employees (identified by ssn, with salary and phone as attributes); departments (identified by dno, with dname and budget as attributes); and children of employees (with name and age as attributes). Employees work in departments; each department is managed by an employee; a child must be identified uniquely by name when the

...Fundamentals of Database System Concepts

parent (who is an employee; assume that only one parent works for the company) is known. We are not interested in information about a child once the parent leaves the company.

Draw an ER diagram that captures this information.

5. Construct appropriate tables for question 4 above.

CHAPTER FOUR

RELATIONAL MODEL

INTRODUCTION

The relational model is very simple and elegant. A database is a collection of one or more relations, where each relation is a table with rows and columns. This simple tabular representation enables even novice users to understand the contents of a database, and it permits the use of simple, high-level languages to query the data. A relational model usually has the following attributes:

- ❖ Relational model stores the data in the form of a table.
- ❖ A single database can be spread across several tables.
- ❖ A relational model uses tables to organize data elements.
- ❖ Each table corresponds to an application entity and each row represents an instance of that entity.

THE RELATIONAL MODEL

The main construct for representing data in the relational model is a relation. A **relation** consists of a **relation schema** and a **relation instance**. The relation instance is a table, and the relation schema describes the column heads for the table. The schema specifies the relation's name, the name of each **field** (or **column**, or **attribute**), and the **domain** of each field. A domain is referred to in a relation schema by the **domain name** and has a set of associated **values**. For example, student information in a university database:

Students (sid: string,
name: string,
login: string,
age: integer,
gpa: real)

This means, for instance, that the field named *sid* has a domain named *string*. The set of values associated with domain *string* is the set of all character strings. An **instance** of a relation is a set of **tuples**, also called **records**, in which each tuple has the same number of fields as the relation schema. A relation instance can be thought of as a table in which each tuple is a row, and all rows have the same number of fields. (The term relation instance is often abbreviated to just **relation**, when there is no confusion with other aspects of a relation such as its schema.)

Today, the relational model is the primary data model for commercial data-processing applications. It has attained its primary position because of its simplicity, which eases the job of the programmer, as compared to earlier data models such as the network model or the hierarchical model.

STRUCTURE OF RELATIONAL DATABASES

A relational database consists of a collection of **tables**, each of which is assigned a unique name. Each table has a structure containing rows and columns. A row in a table represents a *relationship* among a set of values. Since a table is a collection of such relationships, there is a close correspondence between the concept of *table* and the mathematical concept of *relation*, from which the relational data model takes its name.

Consider the *account* table of Figure 4.1. It has three column headers: *account-number*, *branch-name*, and *balance*. Following the terminology of the relational model, we refer to these headers as **attributes**. For each attribute, there is a set of permitted values, called the **domain** of that attribute. For the attribute *branch-name*, for example, the domain is the set of all branch names. Let D_1 denote the

...Fundamentals of Database System Concepts

set of all account numbers, D_2 the set of all branch names, and D_3 the set of all balances. In general, *account* will contain only a subset of the set of all possible rows. Therefore, *account* is a subset of $D_1 \times D_2 \times D_3$

<i>account-number</i>	<i>branch-name</i>	<i>balance</i>
500270-1234	Ilorin	500
500270-1235	Ikeja	400
500270-1236	Osogbo	900
500270-1237	Ibadan	700
500270-1238	Offa	750
500270-1239	Abeokuta	700
500270-1240	Oyo	350

Figure 4.1: The account relation

In general, a **table** of n attributes must be a subset of

$$D_1 \times D_2 \times \cdots \times D_{n-1} \times D_n$$

Mathematicians define a **relation** to be a subset of a Cartesian product of a list of domains. This definition corresponds almost exactly with our definition of *table*. The only difference is that we have assigned names to attributes, whereas mathematicians rely on numeric “names,” using the integer 1 to denote the attribute whose domain appears first in the list of domains, 2 for the attribute whose domain appears second, and so on.

<i>account-number</i>	<i>branch-name</i>	<i>balance</i>
500270-1234	Ilorin	500
500270-1237	Ibadan	700
500270-1238	Offa	750
500270-1239	Abeokuta	700
500270-1235	Ikeja	400
500270-1240	Oyo	350
500270-1236	Osogbo	900

Figure 4.2: The account relation with unordered tuples

Because tables are essentially relations, we shall use the mathematical terms **relation** and **tuple** in place of the terms **table** and **row**. A **tuple variable** is a variable that stands for a tuple; in other words, a tuple variable is a variable whose domain is the set of all tuples.

In the *account* relation of Figure 4.1, there are seven tuples. Let the tuple variable t refer to the first tuple of the relation. We use the notation $t[\textit{account-number}]$ to denote the value of t on the *account-number* attribute. Thus, $t[\textit{account-number}] = \text{“500270-1234,”}$ and $t[\textit{branch-name}] = \text{“Ilorin”}$. Alternatively, we may write $t[1]$ to denote the value of tuple t on the first attribute (*account-number*), $t[2]$ to denote *branch-name*, and so on. Since a relation is a set of tuples, we use the mathematical notation of $t \sqsubset r$ to denote that tuple t is in relation r .

The order in which tuples appear in a relation is irrelevant, since a relation is a *set* of tuples. Thus, whether the tuples of a relation are listed in sorted order, as in Figure 4.1, or are unsorted, as in Figure 4.2, does not matter; the relations in the two figures above are the same, since both contain the same set of tuples.

We require that, for all relations r , the domains of all attributes of r be atomic. A domain is **atomic** if elements of the domain are considered to be indivisible units. For example, the set of integers is an

...Fundamentals of Database System Concepts

atomic domain, but the set of all sets of integers is a non-atomic domain. The distinction is that we do not normally consider integers to have subparts, but we consider sets of integers to have subparts—namely, the integers composing the set. The important issue is not what the domain itself is, but rather how we use domain elements in our database. The domain of all integers would be non-atomic if we considered each integer to be an ordered list of digits.

It is possible for several attributes to have the same domain. For example, suppose that we have a relation *customer* that has the three attributes *customer-name*, *customer-street*, and *customer-city*, and a relation *employee* that includes the attribute *employee-name*. It is possible that the attributes *customer-name* and *employee-name* will have the same domain: the set of all person names, which at the physical level is the set of all character strings. The domains of *balance* and *branch-name*, on the other hand, certainly ought to be distinct. It is perhaps less clear whether *customer-name* and *branch-name* should have the same domain. At the physical level, both customer names and branch names are character strings. However, at the logical level, we may want *customer-name* and *branch-name* to have distinct domains.

One domain value that is a member of any possible domain is the **null** value, which signifies that the value is unknown or does not exist. For example, suppose that we include the attribute *telephone-number* in the *customer* relation. It may be that a customer does not have a telephone number, or that the telephone number is unlisted. We would then have to resort to null values to signify that the value is unknown or does not exist. Null values cause a number of difficulties when we access or update the database, and thus should be eliminated if at all possible.

DATABASE SCHEMA

When we talk about a database, we must differentiate between the database schema, which is the logical design of the database, and a database instance, which is a snapshot of the data in the database at a given instant in time. The concept of a relation corresponds to the programming-language notion of a variable. The concept of a relation schema corresponds to the programming-language notion of type definition. It is convenient to give a name to a relation schema, just as we give names to type definitions in programming languages. We adopt the convention of using lowercase names for relations, and names beginning with an uppercase letter for relation schemas. Following this notation, we use *Account-schema* to denote the relation schema for relation *account*. Thus,

Account-schema = (*account-number*, *branch-name*, *balance*)

We denote the fact that *account* is a relation on *Account-schema* by

account(*Account-schema*)

In general, a relation schema consists of a list of attributes and their corresponding domains. The concept of a **relation instance** corresponds to the programming language notion of a value of a variable. The value of a given variable may change with time; similarly the contents of a relation instance may change with time as the relation is updated. However, we often simply say “*relation*” when we actually mean “*relation instance*.” As an example of a relation instance, consider the *branch* relation of Figure 4.3. The schema for that relation is

Branch-schema = (*branch-name*, *branch-city*, *assets*)

Note that the attribute *branch-name* appears in both *Branch-schema* and *Account-schema*. This duplication is not a coincidence. Rather, using common attributes in relation schemas is one way of relating tuples of distinct relations.

<i>branch-name</i>	<i>branch-city</i>	<i>assets</i>
Taiwo	Ilorin	7100000
Gbodija	Ibadan	9000000
Olofa way	Offa	400000
Olowu	Abeokuta	3700000
Eko	Ikeja	1700000
Muritala	Ilorin	300000
Ataoja	Osogbo	2100000
Unity	Ilorin	8000000

Figure 4.3: The branch relation

For example, suppose we wish to find the information about all of the accounts maintained in branches located in Ilorin. We look first at the *branch* relation to find the names of all the branches located in Ilorin. Then, for each such branch, we would look in the *account* relation to find the information about the accounts maintained at that branch. This is not surprising-recall that the primary key attributes of a strong entity set appear in the table created to represent the entity set, as well as in the tables created to represent relationships that the entity set participates in. Let us continue our banking example. We need a relation to describe information about customers. The relation schema is

Customer-schema = (*customer-name*, *customer-street*, *customer-city*)

Figure 4.4 shows a sample relation *customer* (*Customer-schema*). We assume that the customer name uniquely identifies a customer-obviously this may not be true in the real world, but the assumption makes the examples much easier to read.

<i>Customer-name</i>	<i>Customer-street</i>	<i>Customer-city</i>
Shola	Muritala	Ilorin
Abdullahi	Shango	Kano
Okechukwu	Chinedu	Abia.
Omosetan	Gbodija	Ibadan.
Ishola	Island	Lagos
Akanbi	Oyo	Ibadan
Samuel	Ijebu	Abeokuta
Damilola	Olofa	Offa
Ayodele	Taiwo	Ilorin
Danjuma	Kanuri	Kano
Omoniyi	Mile12	Lagos
Jalingo	Ota	Abeokuta

Figure 4.4: The customer relation

In a real-world database, the *customer-id* (an identifier generated by the bank) would serve to uniquely identify customers. We also need a relation to describe the association between customers and accounts. The relation schema to describe this association is

Depositor-schema = (*customer-name*, *account-number*)

Figure 4.5 shows a sample relation *depositor* (*Depositor-schema*). It would appear that, for our banking example, we could have just one relation schema, rather than several. That is, it may be easier

...Fundamentals of Database System Concepts

for a user to think in terms of one relation schema, rather than in terms of several. Suppose that we used only one relation for our example, with schema;

(*branch-name, branch-city, assets, customer-name, customer-street customer-city, account-number, balance*)

Observe that, if a customer has several accounts, we must list the address once for each account. That is, we must repeat certain information several times. This repetition is wasteful and is avoided by the use of several relations, as in our example.

In addition, if a branch has no accounts (a newly created branch, that has no customers yet), we cannot construct a complete tuple on the preceding single relation, because no data concerning *customer* and *account* are available yet. To represent incomplete tuples, we must use *null* values that signify that the value is unknown or does not exist. Thus, in our example, the values for *customer-name*, *customer-street*, and so on must be null. By using several relations, we can represent the branch information for a bank with no customers without using null values. We simply use a tuple on *Branch-schema* to represent the information about the branch, and create tuples on the other schemas only when the appropriate information becomes available. We include two additional relations to describe data about loans maintained in the various branches in the bank:

<i>customer-name</i>	<i>Account-number</i>
Abdullahi	500270-1234
Abdullahi	500270-1238
Akanbi	500270-1236
Gbolahan	500270-1235
Okechukwu	500270-1239
Omosetan	500270-1240
Shola	500270-1237

Figure 4.5: The depositor relation

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>
L-1	Taiwo	900
L-5	Gbodija	1500
L-7	Olofa way	1500
L-8	Olowu	1300
L-11	Eko	1000
L-13	Muritala	2000
L-15	Ataoja	500

Figure 4.6: The loan relation

SCHEMA DIAGRAM

A database schema, along with primary key and foreign key dependencies, can be depicted pictorially by **schema diagrams**. Figure 4.7 shows the schema diagram for our banking enterprise. Each relation appears as a box, with the attributes listed inside it and the relation name above it. If there are primary key attributes, a horizontal line crosses the box, with the primary key attributes listed above the line.

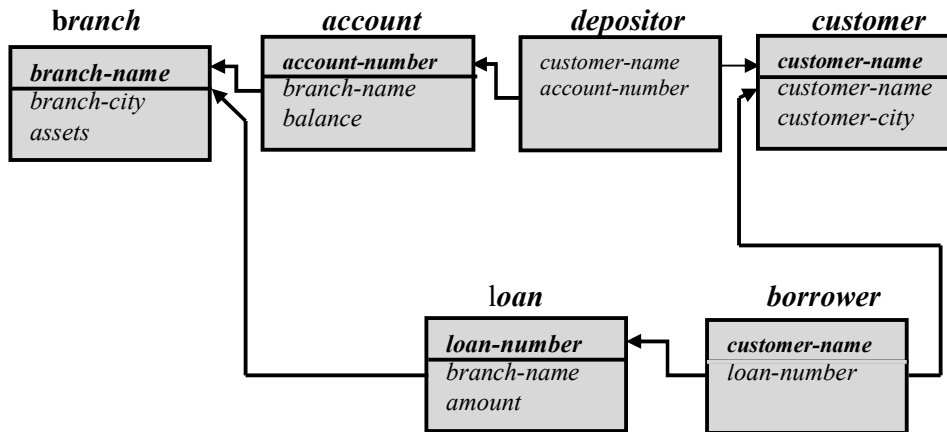


Figure 4.7: Schema diagram for the banking enterprises

Foreign key dependencies appear as arrows from the foreign key attributes of the referencing relation to the primary key of the referenced relation. Do not confuse a schema diagram with an E-R diagram. In particular, E-R diagrams do not show foreign key attributes explicitly, whereas schema diagrams show them explicitly. Many database systems provide design tools with a graphical user interface for creating schema diagrams.

EXERCISE 4

1. Describe the differences in meaning between the terms *relation* and *relation schema*.
2. Design a relational database for a university registrar's office. The office maintains data about each class, including the instructor, the number of students enrolled, and the time and place of the class meetings. For each student-class pair, a grade is recorded.
3. Differentiate between E-R diagram and Schema diagram.

CHAPTER FIVE RELATIONAL DATABASES

INTRODUCTION TO STRUCTURED QUERY LANGUAGE

IBM developed the original version of SQL at its San Jose Research Laboratory (now the Almaden Research Center). IBM implemented the language, originally called Sequel, as part of the System R project in the early 1970s. The Sequel language has evolved since then, and its name has changed to SQL (Structured Query Language). Many products now support the SQL language. SQL has clearly established itself as *the* standard relational-database language.

In 1986, the American National Standards Institute (ANSI) and the International Organization for Standardization (ISO) published an SQL standard, called SQL-86. IBM published its own corporate SQL standard, the Systems Application Architecture Database Interface (SAA-SQL) in 1987. ANSI published an extended standard for SQL, SQL-89, in 1989. The next version of the standard was SQL-92 standard, and the most recent version is SQL:1999. The bibliographic notes provide references to these standards.

The SQL language has several parts:

- i. **Data-definition language (DDL).** The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- ii. **Interactive data-manipulation language (DML).** The SQL DML includes a query language based on both the relational algebra and the tuple relational calculus. It includes also commands to insert tuples into, delete tuples from, and modify tuples in the database.
- iii. **View definition.** The SQL DDL includes commands for defining views.
- iv. **Transaction control.** SQL includes commands for specifying the beginning and ending of transactions.
- v. **Embedded SQL and dynamic SQL.** Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, Java, PL/I, Cobol, Pascal, and Fortran.
- vi. **Integrity.** The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- vii. **Authorization.** The SQL DDL includes commands for specifying access rights to relations and views.

CREATING AND MODIFYING RELATIONS USING SQL

The SQL-92 language standard uses the word *table* to denote relation, and we will often follow this convention when discussing SQL. The subset of SQL that supports the creation, deletion, and modification of tables is called the Data Definition Language (DDL).

Basic Structure

A relational database consists of a collection of relations, each of which is assigned a unique name. Each relation has a structure. SQL allows the use of null values to indicate that the value either is unknown or does not exist. It allows a user to specify which attributes cannot be assigned null values. The basic structure of an SQL expression consists of three clauses: **select**, **from**, and **where**.

- i. The **select** clause is used to list the attributes desired in the result of a query.
- ii. The **from** clause lists the relations to be scanned in the evaluation of the expression.

- iii. The **where** clause consists of a predicate involving attributes of the relations that appear in the **from** clause.

A typical SQL query has the form

```
select  $A_1, A_2, \dots, A_n$ 
from  $r_1, r_2, \dots, r_m$ 
where  $P$ 
```

The select Clause

The select statement is used to retrieve specific information from a table. Let us consider a simple query using banking example, “Find the names of all branches in the *loan* relation”:

```
SELECT branch-name
FROM loan
```

The result is a relation consisting of a single attribute with the heading *branch-name*.

SQL allows us to use the keyword **all** to specify explicitly that duplicates are not removed:

```
SELECT ALL branch-name
FROM loan
```

The asterisk symbol “ * ” can be used to denote “all attributes.” Thus, the use of *loan.** in the preceding **select** clause would indicate that all attributes of *loan* are to be selected. A select clause of the form **select** * indicates that all attributes of all relations appearing in the **from** clause are selected.

The **select** clause may also contain arithmetic expressions involving the operators +, −, □, and / operating on constants or attributes of tuples. For example, the query

```
SELECT loan-number, branch-name, amount * 100
FROM loan
```

will return a relation that is the same as the *loan* relation, except that the attribute *amount* is multiplied by 100.

The where Clause

Let us consider the query “Find all loan numbers for loans made at the Offa branch with loan amounts greater than ₦1200.” This query can be written in SQL as:

```
SELECT loan-number
FROM loan
WHERE branch-name = 'Offa' AND amount > 1200
```

SQL uses the logical connectives **and**, **or**, and **not**—rather than the mathematical symbols $\wedge$, $\vee$, and $\neg$ —in the **where** clause. The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>.

SQL allows us to use the comparison operators to compare strings and arithmetic expressions, as well as special types, such as date types. SQL includes a **between** comparison operator to simplify **where** clauses that specify that a value be less than or equal to some value and greater than or equal to some

...*Fundamentals of Database System Concepts*

other value. If we wish to find the loan number of those loans with loan amounts between ₦90,000 and ₦100,000, we can use the **between** comparison to write

```
SELECT loan-number
FROM loan
WHERE amount BETWEEN 90000 AND 100000
```

instead of

```
SELECT loan-number
FROM loan
WHERE amount <= 100000 AND amount >= 90000
```

Similarly, we can use the **not between** comparison operator.

The Rename Operation

SQL provides a mechanism for renaming both relations and attributes. It uses the **as** clause, taking the form:

old-name as new-name

For example, if we want the attribute name *loan-number* to be replaced with the name *loan-id*, we can rewrite the preceding query as

```
SELECT customer-name, borrower.loan-number AS loan-id, amount
FROM borrower, loan
WHERE borrower.loan-number = loan.loan-number
```

The Create Table Clause

The CREATE TABLE statement is used to define a new table. To create the Students relation, we can use the following statement:

```
CREATE TABLE Studentrec (
  matricno CHAR(20),
  names CHAR(30),
  login CHAR(20),
  age INTEGER,
  gpa REAL);
```

The Insert Clause

Tuples are inserted using the INSERT command. The insert clause can be used to insert or add row (tuple) into a table. We can insert a single tuple into the Students table as follows:

```
INSERT
INTO Students (matricno, name, login, age, gpa)
VALUES ('HND/01/COM/002', 'Smith', 'smith@ee', 18, 3.2)
```

We can optionally omit the list of column names in the INTO clause and list the values in the appropriate order, but it is good style to be explicit about column names.

The Delete Clause

We can delete tuples using the DELETE command. We can delete all Students tuples with name equal to Smith using the command:

```
DELETE
FROM Students
WHERE name = 'Smith'
```

Here are other examples of SQL delete requests:

- (i) Delete all account tuples in the Offa branch.

```
DELETE FROM account
WHERE branch-name = 'Offa'
```

- (ii) Delete all loans with loan amounts between ₦1300 and ₦1500.

```
DELETE FROM loan
WHERE amount BETWEEN 1300 AND 1500
```

- (iii) Delete all account tuples at every branch located in Ilorin.

```
DELETE FROM account
WHERE branch-name IN (SELECT branch-name
FROM branch
WHERE branch-city = 'Ilorin')
```

This **delete** request first finds all branches in Ilorin, and then deletes all *account* tuples pertaining to those branches.

The Update Clause

You can modify the column values in an existing row using the UPDATE command. For example, we can increment the age and decrement the gpa of the student with sid 002:

```
UPDATE Students
SET age = age + 1, gpa = gpa - 1
WHERE sid = 002
```

These examples illustrate some important points. The WHERE clause is applied first and determines which rows are to be modified. The SET clause then determines how these rows are to be modified. If the column that is being modified is also used to determine the new value, the value used in the expression on the right side of equals (=) is the old value, that is, before the modification. To illustrate these points further, consider the following variation of the previous query:

```
UPDATE Students
SET gpa = gpa - 0.1
WHERE gpa >= 3.3
```

Another example, if interest is to be paid only to accounts with a balance of ₦1000 or more, we can write

```
UPDATE account
```

...Fundamentals of Database System Concepts

SET *balance* = *balance* * 1.05

WHERE *balance* >= 1000

In general, the **where** clause of the **update** statement may contain any construct legal in the **where** clause of the **select** statement (including nested **selects**). As with **insert** and **delete**, a nested **select** within an **update** statement may reference the relation that is being updated. As before, SQL first tests all tuples in the relation to see whether they should be updated, and carries out the updates afterward.

INTEGRITY CONSTRAINTS OVER RELATIONS

A database is only as good as the information stored in it, and a DBMS must therefore help prevent the entry of incorrect information. An **integrity constraint (IC)** is a condition that is specified on a database schema, and restricts the data that can be stored in an instance of the database. If a database instance satisfies all the integrity constraints specified on the database schema, it is a legal instance. A DBMS enforces integrity constraints, in that it permits only legal instances to be stored in the database. Integrity constraints are specified and enforced at different times:

1. When the DBA or end user defines a database schema, he or she specifies the ICs that must hold on any instance of this database.
2. When a database application is run, the DBMS checks for violations and disallows changes to the data that violate the specified ICs. In some situations, rather than disallow the change, the DBMS might instead make some compensating changes to the data to ensure that the database instance satisfies all ICs. In any case, changes to the database are not allowed to create an instance that violates any IC.

Many kinds of integrity constraints can be specified in the relational model. Some of the integrity constraints are: *key constraints, foreign key constraints and general key constraints*.

Key constraints

Consider the Students relation and the constraint that no two students have the same student id. This IC is an example of a **key constraint**. A key constraint is a statement that a certain minimal subset of the fields of a relation is a unique identifier for a tuple. A set of fields that uniquely identifies a tuple according to a key constraint is called a **candidate key** for the relation; we often abbreviate this to just key. In the case of the Students relation, the (set of fields containing just the) sid field is a candidate key.

Let us take a closer look at the above definition of a (candidate) key. There are two parts to the definition:

1. Two distinct tuples in a legal instance (an instance that satisfies all ICs, including the key constraint) cannot have identical values in all the fields of a key.
2. No subset of the set of fields in a key is a unique identifier for a tuple. The first part of the definition means that in any legal instance, the values in the key fields uniquely identify a tuple in the instance.

When specification key constraint, the DBA or user must be sure that this constraint will not prevent them from storing a correct' set of tuples. (A similar comment applies to the specification of other kinds of ICs as well.) The notion of 'correctness' here depends upon the nature of the data being stored. For example, several students may have the same name, although each student has a unique student id. If the name field is declared to be a key, the DBMS will not allow the Students relation to contain two tuples describing different students with the same name.

For example, in table definition and specify key information:

```
CREATE TABLE Students (  
  sid CHAR(20),  
  name CHAR(30),  
  login CHAR(20),  
  age INTEGER,  
  gpa REAL,  
  UNIQUE (name, age),  
  CONSTRAINT Students Key PRIMARY KEY (sid)
```

This definition says that *sid* is the primary key and that the combination of *name* and *age* is also a key. The definition of the primary key also illustrates how we can name a constraint by preceding it with **CONSTRAINT** constraint-name. If the constraint is violated, the constraint name is returned and can be used to identify the error.

Foreign key constraints

Sometimes the information stored in a relation is linked to the information stored in another relation. If one of the relations is modified, the other must be checked, and perhaps modified, to keep the data consistent. An IC involving both relations must be specified if a DBMS is to make such checks. The most common IC involving two relations is a foreign key constraint.

Suppose that in addition to *Students*, we have a second relation: *Enrolled* (*sid*: string, *cid*: string, *grade*: string) To ensure that only bona fide students can enrol in courses, any value that appears in the *sid* field of an instance of the *Enrolled* relation should also appear in the *sid* field of some tuple in the *Students* relation. The *sid* field of *Enrolled* is called a foreign key and refers to *Students*.

The foreign key in the referencing relation (*Enrolled*, in our example) must match the primary key of the referenced relation (*Students*), i.e., it must have the same number of columns and compatible data types, although the column names can be different.

This constraint is illustrated in the figure showing below, there may well be some students who are not referenced from *Enrolled* (e.g., the student with *sid* = 50000). However, every *sid* value that appears in the instance of the *Enrolled* table appears in the primary key column of a row in the *Students* table.

Foreign key

<i>cid</i>	<i>grade</i>	<i>sid</i>
COM 111	C	001
COM 112	B	002
COM 114	A	003
COM 115	B	004

Primary key

<i>sid</i>	<i>name</i>	<i>login</i>	<i>age</i>	<i>gpa</i>
005	Dave	dave@cs	19	3.3
004	Jones	jones@cs	18	3.4
006	Smith	smith@ee	18	3.2
003	Smith	smith@math	19	3.8
001	Madayan	madayan@music	11	1.8
002	Guldu	guldu@music	12	2.0

If we try to insert the tuple 009, COM 117, the IC is violated because there is no tuple in SI with the id 009; the database system should reject such an insertion. Similarly, if we delete the tuple 010, Jones, jones@cs, 18, 3.4 from SI, we violate the foreign key constraint because the tuple 010, COM 115,

...Fundamentals of Database System Concepts

contains sid value 004, the sid of the deleted Students tuple. The DBMS should disallow the deletion or, perhaps, also delete the Enrolled tuple that refers to the deleted Students tuple.

General Constraints

Domain, primary key, and foreign key constraints are considered to be a fundamental part of the relational data model and are given special attention in most commercial systems. Sometimes, however, it is necessary to specify more general constraints. For example, we may require that student ages be within a certain range of values; given such an IC specification, the DBMS will reject inserts and updates that violate the constraint. This is very useful in preventing data entry errors.

If we specify that all students must be at least 16 years old, the instance of Students shown in table above is illegal because two students are underage (001 and 002). The IC that students must be older than 16 can be thought of as an extended domain constraint, since we are essentially defining the set of permissible age values more stringently than is possible by simply using a standard domain such as integer. In general, however, constraints that go well beyond domain, key, or foreign key constraints can be specified. For example, we could require that every student whose age is greater than 18 must have a gpa greater than 3.

Current relational database systems support such general constraints in the form of table constraints and assertions. Table constraints are associated with a single table and are checked whenever that table is modified. In contrast, assertions involve several tables and are checked whenever any of these tables is modified.

ENFORCING INTEGRITY CONSTRAINTS

As we observed earlier, ICs are specified when a relation is created and enforced when a relation is modified. The impact of domain, PRIMARY KEY, and UNIQUE constraints is straightforward: if an insert, delete, or update command causes a violation, it is rejected. Potential IC violation is generally checked at the end of each SQL statement execution, although it can be deferred until the end of the transaction executing the statement.

Consider the instance of Students shown in table above. The following insertion violates the primary key constraint because there is already a tuple with the sid 006, and it will be rejected by the DBMS:

```
INSERT
INTO Students (sid, name, login, age, gpa)
VALUES (006, 'Mike', 'mike@ee', 17,3.4)
```

The following insertion also violates the constraint that the primary key cannot contain null:

```
INSERT
INTO Students (sid, name, login, age, gpa)
VALUES (null, 'Mike', 'mike@ee', 17,3.4)
```

QUERYING RELATIONAL DATA

A relational database query is a question about the data, and the answer consists of a new relation containing the result. For example, we might want to find all students younger than 18 or all students enrolled in COM112. A query language is a specialized language for writing queries. SQL is the most popular commercial query language for a relational DBMS. We now present some SQL examples that illustrate how easily relations can be queried. Consider the instance of the Students relation shown

table above. We can retrieve rows corresponding to students who are younger than 18 with the following SQL query:

```
SELECT *  
FROM Students  
WHERE age < 18
```

EXERCISE 5

1. Discuss your understanding by the term relational database.
2. Describe the function of any six SQL statements.
3. Consider the relational database below. Give an expression in SQL for each of the following queries.

employee (*employee-name*, *street*, *city*)
works (*employee-name*, *company-name*, *salary*)
company (*company-name*, *city*)
manages (*employee-name*, *manager-name*)

- a. Modify the database so that Ayodele now lives in Ilorin.
 - b. Give all employees of First Bank Corporation a 10 percent raise.
 - c. Give all managers of First Bank Corporation a 10 percent raise.
 - d. Give all managers of First Bank Corporation a 10 percent raise unless the salary becomes greater than ₦100,000; in such cases, give only a 3 percent raise.
 - e. Delete all tuples in the *works* relation for employees of Small Bank Corporation.
4. Use the relational database schema below to answer the following questions:

Emp(*eid*: integer, *ename*: string, *age*: integer, *salary*: real)
Works(*eid*: integer, *did*: integer, *pct time*: integer)
Dept(*did*: integer, *dname*: string, *budget*: real, *managerid*: integer)

- a. Give an example of a foreign key constraint that involves the *Dept* relation. What are the options for enforcing this constraint when a user attempts to delete a *Dept* tuple?
 - b. Write the SQL statements required to create the above relations, including appropriate versions of all primary and foreign key integrity constraints.
 - c. Define the *Dept* relation in SQL so that every department is guaranteed to have a manager.
 - d. Write an SQL statement to add 'John Doe' as an employee with
 - e. *eid* = 101, *age* = 32 and *salary* = 15; 000.
 - f. Write an SQL statement to give every employee a 10% raise.
5. Write an SQL statement to delete the 'Toy' department. Given the referential integrity constraints you chose for this schema, explain what happens when this statement is executed.
 6. Discuss the need for enforcing Integrity constraints.

CHAPTER SIX DATABASE DESIGN ARCHITECTURE

INTRODUCTION

The architecture of a database system is greatly influenced by the underlying computer system on which the database system runs. Database systems can be centralized, or client–server, where one server machine executes work on behalf of multiple client machines. Database systems can also be designed to exploit parallel computer architectures. Distributed databases span multiple geographically separated machines. Centralized database management systems, is one in which all the data is maintained at a single site, and assumed that the processing of individual transactions is essentially sequential. One of the most important trends in databases is the increased use of parallel evaluation techniques and data distribution.

ARCHITECTURES FOR PARALLEL DATABASES

A **parallel database system** is one that seeks to improve performance through parallel implementation of various operations such as loading data, building indexes, and evaluating queries. Although data may be stored in a distributed fashion in such a system, the distribution is governed solely by performance considerations. Three main architectures have been proposed for building parallel DBMSs. In a **shared-memory** system, multiple CPUs are attached to an interconnection network and can access a common region of main memory. In a **shared-disk** system, each CPU has a private memory and direct access to all disks through an interconnection network. In a **shared-nothing** system, each CPU has local main memory and disk space, but no two CPUs can access the same storage area; all communication between CPUs is through a network connection. The three architectures are illustrated in Figure 6.1.

The shared memory architecture is closer to a conventional machine, and many commercial database systems have been ported to shared memory platforms with relative ease. Communication overheads are low, because main memory can be used for this purpose, and operating system services can be leveraged to utilize the additional CPUs.

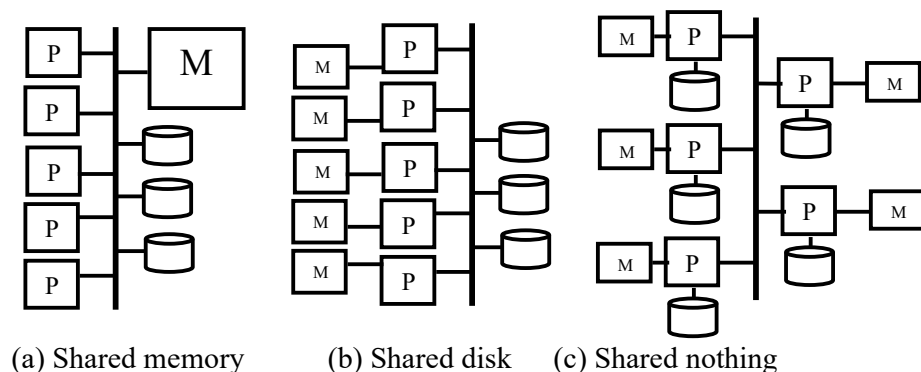


Figure 6.1: Physical Architectures for Parallel Database Systems

Although this approach is attractive for achieving moderate parallelism, a few tens of CPUs can be exploited, in this fashion memory contention becomes a bottleneck as the number of CPUs increases. The shared-disk architecture faces a similar problem because large amounts of data are shipped through the interconnection network. The basic problem with the **shared-memory** and **shared-disk** architectures is **interference**: As more CPUs are added, existing CPUs are slowed down because of the increased contention for memory accesses and network bandwidth.

The **shared-nothing** architecture requires more extensive reorganization of the DBMS code, but it has been shown to provide linear **speed-up**, in that the time taken for operations decreases in proportion to

...Fundamentals of Database System Concepts

the increase in the number of CPUs and disks, and linear **scale-up**, in that performance is sustained if the number of CPUs and disks are increased in proportion to the amount of data. Consequently, ever-more powerful parallel database systems can be built by taking advantage of rapidly improving performance for single CPU systems and connecting as many CPUs as desired.

DISTRIBUTED DATABASES SYSTEM

In a distributed database system, data is physically stored across several sites, and each site is typically managed by a DBMS that is capable of running independently of the other sites. The location of data items and the degree of autonomy of individual sites have a significant impact on all aspects of the system, including query optimization and processing, concurrency control, and recovery. In contrast to parallel databases, the distribution of data is governed by factors such as local ownership and increased availability, in addition to performance issues. Data in a distributed database system is stored across several sites and each site is typically managed by a DBMS that can run independently of the other sites. The classical view of a distributed database system is that the system should make the impact of data distribution **transparent**. In particular, the following properties are considered desirable:

Distributed data independence

Users should be able to ask queries without specifying where the referenced relations, or copies or fragments of the relations, are located. This principle is a natural extension of physical and logical data independence. Further, queries that span multiple sites should be optimized systematically in a cost-based manner, taking into account communication costs and differences in local computation costs.

Distributed transaction atomicity

Users should be able to write transactions that access and update data at several sites just as they would write transactions over purely local data. In particular, the effects of a transaction across sites should continue to be atomic; that is, all changes persist if the transaction commits, and none persist if it aborts.

Although most people would agree that the above properties are in general desirable, in certain situations, such as when sites are connected by a slow long-distance network, these properties are not efficiently achievable. Indeed, it has been argued that when sites are globally distributed, these properties are not even desirable. The argument essentially is that the administrative overhead of supporting a system with distributed data independence and transaction atomicity in effect, coordinating all activities across all sites in order to support the view of the whole as a unified collection of data is prohibitive, over and above DBMS performance considerations.

TYPES OF DISTRIBUTED DATABASES

If data is distributed but all servers run the same DBMS software, we have a **homogeneous distributed database system**. If different sites run under the control of different DBMSs, essentially autonomously, and are connected somehow to enable access to data from multiple sites, we have a heterogeneous distributed database system, also referred to as a multi database system.

DISTRIBUTED DBMS ARCHITECTURES

There are three alternative approaches to separating functionality across different DBMS-related processes; these alternative distributed DBMS architectures are called Client-Server, Collaborating Server, and Middleware.

Client-Server Systems

A Client-Server system has one or more client processes and one or more server processes, and a client process can send a query to anyone server process. Clients are responsible for user-interface issues, and servers manage data and execute transactions.

...Fundamentals of Database System Concepts

Thus, a client process could run on a personal computer and send queries to a server running on a mainframe. This architecture has become very popular for several reasons. First, it is relatively simple to implement due to its clean separation of functionality and because the server is centralized. Second, expensive server machines are not underutilized by dealing with mundane user-interactions, which are now relegated to inexpensive client machines. Third, users can run a graphical user interface that they are familiar with, rather than the (possibly unfamiliar and unfriendly) user interface on the server. While writing Client-Server applications, it is important to remember the boundary between the client and the server and to keep the communication between them as set-oriented as possible. In particular, opening a cursor and fetching tuples one at a time generates many messages and should be avoided. (Even if we fetch several tuples and cache them at the client, messages must be exchanged when the cursor is advanced to ensure that the current row is locked.) Techniques to exploit client-side caching to reduce communication overhead have been extensively studied, although we will not discuss them further.

Collaborating Server Systems

The Client-Server architecture does not allow a single query to span multiple servers because the client process would have to be capable of breaking such a query into appropriate subqueries to be executed at different sites and then piecing together the answers to the subqueries. The client process would thus be quite complex, and its capabilities would begin to overlap with the server; distinguishing between clients and servers becomes harder. Eliminating this distinction leads us to an alternative to the Client-Server architecture: a **Collaborating Server** system. We can have a collection of database servers, each capable of running transactions against local data, which cooperatively execute transactions spanning multiple servers.

When a server receives a query that requires access to data at other servers, it generates appropriate subqueries to be executed by other servers and puts the results together to compute answers to the original query. Ideally, the decomposition of the query should be done using cost-based optimization, taking into account the costs of network communication as well as local processing costs.

Middleware Systems

The Middleware architecture is designed to allow a single query to span multiple servers, without requiring all database servers to be capable of managing such multisite execution strategies. It is especially attractive when trying to integrate several legacy systems, whose basic capabilities cannot be extended.

The idea is that we need just one database server that is capable of managing queries and transactions spanning multiple servers; the remaining servers only need to handle local queries and transactions. We can think of this special server as a layer of software that coordinates the execution of queries and transactions across one or more independent database servers; such software is often called middleware. The middleware layer is capable of executing joins and other relational operations on data obtained from the other servers, but typically, does not itself maintain any data.

STORING DATA IN A DISTRIBUTED DBMS

In a distributed DBMS, relations are stored across several sites. Accessing a relation that is stored at a remote site incurs message-passing costs, and to reduce this overhead, a single relation may be partitioned or fragmented across several sites, with fragments stored at the sites where they are most often accessed, or replicated at each site where the relation is in high demand.

Fragmentation

Fragmentation consists of breaking a relation into smaller relations or fragments, and storing the fragments (instead of the relation itself), possibly at different sites. In **horizontal fragmentation**, each fragment consists of a subset of rows of the original relation. In **vertical fragmentation**, each fragment consists of a subset of columns of the original relation. Horizontal and vertical fragments are illustrated in Figure 6.2

TID	eid	name	city	age	sal
t1	53666	Jones	Madras	18	35
t2	53688	Smith	Chicago	18	32
t3	53650	Smith	Chicago	19	48
t4	53831	Madayan	Bombay	11	20
t5	53832	Guldu	Bombay	12	20

Vertical Fragment

Horizontal Fragment

Figure 6.2: Horizontal and Vertical Fragmentation

Typically, the tuples that belong to a given horizontal fragment are identified by a selection query; for example, employee tuples might be organized into fragments by city, with all employees in a given city assigned to the same fragment. The horizontal fragment shown in Figure 6.2 corresponds to Chicago. By storing fragments in the database site at the corresponding city, we achieve locality of reference—Chicago data is most likely to be updated and queried from Chicago, and storing this data in Chicago makes it local (and reduces communication costs) for most queries. Similarly, the tuples in a given vertical fragment are identified by a projection query. The vertical fragment in the figure results from projection on the first two columns of the employees' relation.

When a relation is fragmented, we must be able to recover the original relation from the fragments:

- Horizontal fragmentation: The union of the horizontal fragments must be equal to the original relation. Fragments are usually also required to be disjoint.
- Vertical fragmentation: The collection of vertical fragments should be lossless-join decomposition.

To ensure that a vertical fragmentation is lossless-join, systems often assign a unique tuple id to each tuple in the original relation, as shown in Figure 14.3, and attach this id to the projection of the tuple in each fragment. If we think of the original relation as containing an additional tuple-id field that is a key, this field is added to each vertical fragment. Such decomposition is guaranteed to be lossless-join. In general a relation can be (horizontally or vertically) fragmented, and each resulting fragment can be further fragmented.

Replication

Replication means that we store several copies of a relation or relation fragment. An entire relation can be replicated at one or more sites. Similarly, one or more fragments of a relation can be replicated at other sites. For example, if a relation R is fragmented into R1, R2, and R3, there might be just one copy of R1, whereas R2 is replicated at two other sites and R3 is replicated at all sites.

The motivation for replication is twofold:

1. **Increased availability of data:** If a site that contains a replica goes down, we can find the same data at other sites. Similarly, if local copies of remote relations are available, we are less vulnerable to failure of communication links.
2. **Faster query evaluation:** Queries can execute faster by using a local copy of a relation instead of going to a remote site.

There are two kinds of replication, called synchronous and asynchronous replication, which differ primarily in how replicas are kept current when the relation is modified. In **synchronous replication**, all copies of a replicated relation are updated before the transaction commits. In **asynchronous replication**, copies are only updated periodically.

EXERCISE 6

1. Why is it relatively easy to port a database from a single processor machine to a multiprocessor machine if individual queries need not be parallelized?
2. Define the terms scale-up and speed-up.
3. What are the similarities and differences between parallel and distributed database management systems?
4. Why is a shared-nothing architecture attractive for parallel database systems?
5. Briefly describe and compare the Client-Server and Collaborating Servers architectures.
6. Define the terms fragmentation and replication, in terms of where data is stored.

CHAPTER SEVEN OVERVIEW OF DATABASE PROGRAM

INTRODUCTION

Database program is a computer-based record keeping system, a system whose overall purpose is to record and maintain information. It provides facilities to store and process data in the database. The program can help to store, print out or display on the screen the records or part of record in the appropriate order. The user can make change to the information contained in the record and can also select records according to desired criteria. Usually, application systems within an organization are designed to be independent of each other such that each application system has its own separate files. For instance, the payroll system of many organizations were usually design and developed independent of the personnel record system, even though the systems have some data in common. The result of this approach is that data are duplicated (e.g. Employee's name, Staff ID no, Dept...,Gross Pay, Total Pay etc), and in most cases there is inconsistency of data because the separate file containing the employee's data are not updated at the same time when changes to employee's data arises. Examples of database program on PC are Dbase III plus, Rbase, FoxBASE, beta, TasPlus, Superbase64 etc.

MANAGEMENT OF DATA IN DATABASE PROGRAMS

In database terminology, a record is often referred to as an “**entity**”, while a field is called an “**attribute**”. In addition, application programs are usually made to be aware of physical characteristics of the organization and accessing of the files. Changes made to the organization and accessing technique will necessitate changes to the existing operational application programs.

In a database system, data duplication and data inconsistency are greatly minimized, and the programmer needs not to be aware of the method of organization or accessing technique of the database. This property is referred to as **data independence**. Database programs provide facilities for handling data in the database. Database packages usually provide facilities to perform the following operations:

- a) It defines what and how the data is to be stored and also keep it up-to-date.
- b) It provides the ways in which the stored data is to be retrieved from storage and presented to the user.
- c) It obtains answers to queries about data.
- d) It enters data or update data.

The result of the above facilities can either be displayed on the screen or printed on the paper through the computer printer.

CHARACTERISTICS OF DBASE PROGRAMS

Dbase program differ in the maximum length of field and record that can be allowed in the database; number of field in each record, number of sort key, ease of making change to the database (either to the structure data or the actual data) and speed at which the program executes.

Some of these programs exist in both the **ASSIST** and **PROGRAMMING** mode. The **Assist** mode is primarily designed for non-computer professional while the **Programming** mode can be used to perform many functions which are only formally available in the programming language. These programs vary in some of the following ways:

- a. Some programs require the length of fields and records to be specified in advance; other allows variable fields length and variable record length. **Card box** and **Cornerstone** are examples of database program that permit variable length fields and records. **Dbase IV** required the field to be specified in advance; maximum of 64 fields only is allowed.

...Fundamentals of Database System Concepts

- b. Some programs allow number of field in each record to vary, others do not. In a situation in which we need to store the 'Home address and or Schools attended' by an individual, number of such field will vary in the organization. As a result a program which allows variable number of fields in a record may be suitable. In such a program facility to relate 2 or more different files to each other are usually provided. Dbase provides this facility.
- c. They also vary in the number of **sort** and **indexing** field that can be used by different program.
- d. The relative ease of making changes to the actual stored data or the database structure varies from one program to the other. Changing records and fields is relatively easy in Data gem and Dbase IV.
- e. The speed with which the program runs or operates varies from one program to other.
- f. The amounts of control over reports produced by some programs also vary. Some allow reports to be produced in a form that can be used by other packages such as word processor (e.g. Dbase IV), while others do not permit this feature.
- g. The mode of operation of the database programs also varies. Some are **purely-menu-driven** e.g. Dbase IV, in which a list of options are displayed on the screen and the user is expected to choose only one.
- h. Other programs allow users to write programs like any other high level language. This type of program is called **command-driven** program. Some program provides the 2 mode of operation (e.g. dbase iv).
- i. In a shared database i.e. database that can be accessed by more than one user, level of access and protection features vary from one program to the other.

INTRODUCTION TO DBASE IV PACKAGE

Dbase IV is extensive and sophisticated application software that can be used in many different ways to provide useful information from large amount of data. Dbase is a product of Aston-Tate of U.S.A. and is one of the most popular DBMS package on micro computers. It is both menu-driven and command driven. The menu driven made usually called the “**Assist**” is the default operating mode. It provides a series of user friendly, pull-down menu to help novice users perform various operations, which would otherwise be performed by a frequent user at the command mode. It also supports a programming language of its own which allows advanced programmers to exploit the full potentials of this software.

STARTING DBASE IV

As the Dbase IV package is loaded, press F2 to invoke the Assist mode, if you are not taken to the ASSIST as the program is loaded. The first menu deals with how to create a database when you press enter key, it will request for the name of the database, type the name, the extension (**.Dbf**) is automatically appended.

A new screen is displayed where you are expected to define the data structure under the following heading:

- * **Field Name:** It specified the name of the field
- * **Type:** This is the type of data to occupy the field (i.e. Character, Numeric, Date, Logical, and Memo).
- * **Width:** The length of the character or numeric data.
- * **Dec:** Number of decimal places if it is numeric data.

...Fundamentals of Database System Concepts

For the **type** character is displayed by default, press **space-bar** to change the type. When you finish creating the structure, then save it by pressing Ctrl + End or Ctrl +W, it will display a message;

“Input data record now?” Y/N

Press Y to start input your data records. A new screen with the structure heading will appear. When you finish entering the records, press Ctrl-W or Ctrl + End to save.

Dbase IV provides facilities to perform various operations like indexing, and sorting by selecting the appropriate option from the menu. The package is relatively easy to use because it is highly menu driven.

CREATING A DATABASE

To create a database with name **studfile.dbf**, which will contain information about a student, MATRIC-NO, NAME, LEVEL, SEX, ENTRY DATE and DEPARTMENT. The following steps are required to create the database

- Step1** Choose **CREATE** from the main menu or type **CREATE** on the dot prompt
- Step2** Type **STUDFILE** when it requests for the database name
- Step3** The format for the structure will be displayed
- Step4** For the first field under **File Name** type MATRIC-NO and press **ENTER** key
- Step5** The cursor will automatically move to the next field, for the type of field, press the space bar until **Numeric** is shown, then press **ENTER** key
- Step6** The cursor will move to the **WIDTH** column, then type 5 and press **ENTER** key
- Step7** The cursor will move to the **DEC** column to indicate the number of decimal places, type **0** and press **ENTER** key
- Step8** Repeat steps 1 – 7 for each of the other fields, choosing the appropriate type for each; remember to choose **DATE** for the entry date field. When you finished it will look like this:

FIELD NAME	TYPE	WIDTH	DEC
MATRIC-NO	NUMERIC	5	0
NAME	CHARACTER	15	0
LEVEL	CHARACTER	6	0
SEX	CHARACTER	8	0
ENTRY DATE	DATE	8	
DEPT	CHARACTER	20	0

- Step9** Press CTRL-END or CTRL-W to save the structure.

To enter the data into the database, choose **APPEND** from the main menu or type APPEND on the dot prompt, it will display the structure for you to enter the data, when you finish save the records entered.

To perform operation like **indexing and sorting** use the appropriate option from the main menu and the questions that follows it.

To quit from the Dbase IV package, remember to close the database by choosing **CLOSE** from the menu or type **CLOSE ALL** at the dot prompt, then choose **QUIT** from the menu or type **QUIT** at dot prompt to finally quit from the package.

EXERCISE 7

1. Define the following terms; Database, DBMS, data, record, field and character.
2. Highlight the features that can be used to differentiate one Dbase program from another with examples.
3. Write the steps involved in creating a new database in Dbase IV.

CHAPTER EIGHT

MICROSOFT ACCESS 2000 AND ABOVE

INTRODUCTION

Microsoft Access is a software package by Microsoft Corporation for database. It is a powerful program to create and manage your databases. It has many built-in features for constructing and viewing your information. Access is a relational database-management system. It allows you to store organize and manipulate collections of information. It can be used to store information and sort out information according to relationship. Microsoft Access is management software that provides all the necessary tools to store, retrieve, manipulate and print the stored data. **Data** itself is a raw fact; **access** is a collection of related data. Data may be any of hierarchical, networks or relational, which is based on the way data items is arranged. A relational data is arranged into **rows** and **columns**, the horizontal arrangement is called **record**, while the vertical arrangement is called **field**. Access is much more involved and is a more genuine database application than other programs.

THE DATABASE

Database is a collection of information that related to a particular subject or purpose, such as keeping student's record in an institution. Microsoft Access breaks down a database into **Database file**, **Table**, **Record**, and **Data-type**. The hierarchy that Access uses in breaking down a database is given bellow;

- (i) **Database file:** This is the main file that encompasses the entire database and that is saved to hard-drive or floppy disk. For example, StudentDatabase.mdb.
- (ii) **Table:** A Table is a collection of data about a specific topic. It stores relative data in rows as **records** and column as **fields**. There can be multiple tables in a database. For example Student's Table and Teacher's Table.
- (iii) **Fields:** Fields are the different categories within a Table. They usually contain multiple fields. For example Student Surname and Student First name.
- (iv) **Data Type:** These are the properties of each field. A field can only have one data type. For example Student Department belongs to **Text** data type.

COMMON TERMS USED IN MS-ACCESS

- a. **Table:** Table is a collection of related database stored in rows and columns.
- b. **Query:** It is used to store or retrieve records in a table. It sorts out the required records from table.
- c. **Form:** Forms can be used to display records in a more alternative mode.
- d. **Report:** It is used to file good presentations of records on a query or table. It can also be used to group records.
- e. **Module:** A module is a collection of visual for application declaration and procedures that store together as a unit.
- f. **Macro:** A macro is a series of word commands and instructions that you group together as a simple command to accomplish a task automatically

STARTING MS-ACCESS

There are two common ways to start Ms-Access;

- (A) Double click on the Ms-Access icon on the desktop.
- (B)

Step1	Click on Start
Step2	Select Programs
Step3	Click Microsoft Access

CREATING NEW AND OPENING EXISTING DATABASE

When starting Ms-access, it will display the Window below;

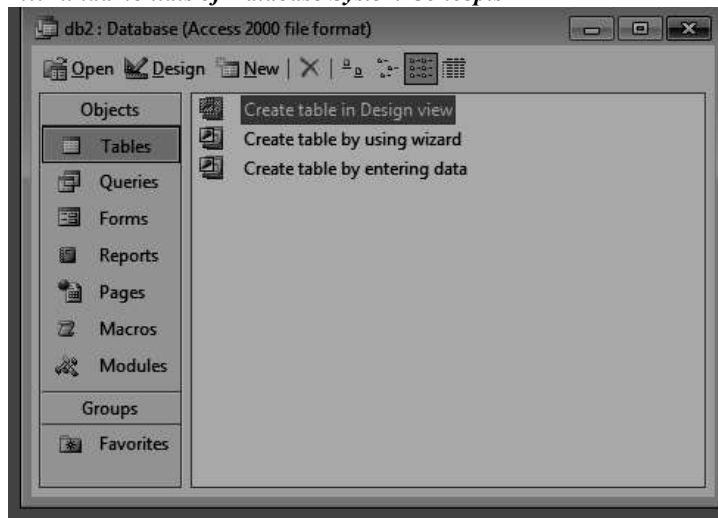


Figure showing Ms-Access Window

The picture above provides the following options;

- Create a new database from scratch.
- Use the wizard to create a new database
- Open an existing database.

I To Create a Database without Using the Database Wizard

Step1 Start up **Ms-Access**. A dialog box is automatically displayed with options.

Step2 Click **Blank Access Database**

Step3 Click **OK**

In case you have already opened a database or close that dialog box, then

Step1 Click **New Database** on the **Toolbar**.

Step2 Double click **Blank Database** icon on the **General tab**

Step3 Specify the name and the location for the database and then

Step4 Click **Create**.

II To Create a Database Using the Database Wizard

Step1 Start up **Ms-Access**. A dialog box is automatically displayed with options

Step2 Click **Access Database Wizards, pages, and projects**

Step3 Click **OK**

In case you have already opened a database or close that dialog box, then

Step1 Click **New Database** on the **Toolbar**

Step2 Double click the kind of database you want to create on the **Database tab**

Step3 Specify the name and the location for the database and then

Step4 Click **Create** to start define your new database.

III To Open an Existing Database.

Step1 Start up **Ms-Access**. A dialog box is automatically displayed with options.

Step2 Click **Open an existing file**. The white box gives the most desired database you have used

Step3 Choose the desired database you had previously used or choose the **More File** option, if you do not see the one had created.

Step4 Click **OK**

TABLES IN MS-ACCESS

A table can be defined as a collection of data about a specific topic, such as employees or students. A separate table can be used for each topic so that you store data only once, which makes the database more efficient, and also reduces data-entry errors.

Table usually organized data into columns (called **fields**) and rows (called **records**). A field in the record table contains the same type of information such as Matriculation No; this is an example of a **COLUMN**. Each record in the record table contain all of the information about an entity such as First Name, Last Name, Birth date, Address, and City etc, this is an example of **ROW**.

CREATING A TABLE USING WIZARD

A table wizard helps in creating a table by offering sample table for different application. Each sample table contains a set of sample fields specific to the application.

- Step1** Double click on **Create Table by Using Wizard** in the database Window.
- Step2** The **Table Wizard** dialog box appears with a list of samples. (Two types of table are **Business table** and **Personal Table**).
- Step3** Select the table of your choice.
- Step4** The sample fields list will change to display field for the newly selected table.
- Step5** Click **File**.
- Step6** Select **Save**
- Step7** Enter the name for your table. Access automatically set the primary key.

TO CREATE A TABLE FROM SCRATCH IN DESIGN VIEW

When creating a table in design view, you have a pointer control over field names and field sizes.

- Step1** Switch to **Database** Window
- Step2** Double click “**Create table in design view**”
- Step3** Define each of the field in your table.
 - a. Enter the categories of your table under the **Field Name column**
 - b. Enter the type you want for your categories under **Data Type column**.

Ms-Access database has the following data type;

1. Text and memo: Allow the field to store text or number.
2. Number: It stores numeric data that will be used in mathematical calculations.
3. Currency: Used to display or calculate currency values.
4. Logical: Is the field containing YES/NO or TRUE/FALSE.
5. Float: Field containing floating point binary number that may be integer or decimal. Other types include Date/Time, Auto Number and OLE object (Picture).

- Step4** Enter the text that describes what each field means (optional). For example,

Field name	Data Type	Description
Matric_No	Text	Matriculation number, uniquely identifies a student
Surname	Text	Student's Surname
Last Name	Text	Student's Last name
Date of Birth	Date/time	Student's Date of Birth
Address	Text	Student's Address

...Fundamentals of Database System Concepts

City	Text	City student resides in
State	Text	State student resides in
G.S.M No	Text	Student's GSM number

PRIMARY KEY

The primary key is one or more fields (columns) whose values uniquely identify each record in a table. A primary key does not allow **Null** values and must always have a unique value. You do not have to define a primary key, but it is usually good idea. If you don't define a primary key, Ms-Access will ask if you would like to create one when you save the table.

Matriculation Number field is the primary key in above table, which simply means that every student has a Matriculation number and no two students are the same. To do this,

- Step1** Select the **Matric_No** field
- Step2** Click the **Primary key** button.?
- Step3** Save the table.

SWITCHING VIEWS

Ms-Access table contains all of your data which are the foundation of your database. There are two major ways to examine the table, **Datasheet view** and **design view**. To switch views from the datasheet and design view, simply click the button in the top left corner of the Access program.

- I Datasheet View:** Displays the view, which allows you to enter raw data into your database table.
- II Design View:** Displays the view, which allows you to enter fields, data-type, and descriptions into your database table.
 - A. Entering Data:** Click on the **Datasheet View** and simply start entering the data into each field. Before starting a new record, the **Matric_No** field must have something in it, because it is the **primary key** except it has not been set.
 - B. Manipulating Data**
 - (i) Adding a new row:** Just drop down to a new line and enter the information.
 - (ii) Updating a record:** Simply select the record and field you want to update, and then change its data with what you want.
 - (iii) Deleting a record:** Just select the entire row and press delete key on the keyboard.

MORE TABLE FEATURES IN MS-ACCESS.

I To Assign a Field To a Specific Set of Character

- Step1** Switch to **Design View**.
- Step2** Select the field you want to alter.
- Step3** Click the **General Tab** at the bottom
- Step4** Select field **size**
- Step5** Enter the number of characters you want this field to have.

II To Format a Field To Look In a Specific Way

- Step1** Switch to **Design view**
- Step2** Select the field you want to format.
- Step3** Click **General Tab** at the bottom
- Step4** Select **Input Mask Box** and then click on the ... button at the right.
- Step5** Select Phone number option.
- Step6** Click on **Next**
- Step7** just leave! **(999) 000-0000**, it is the default.
- Step8** Click **Next**
- Step9** Select which option you want it to look like.

Step10 Click **Next**

Step11 Click **Finish**

RELATIONSHIPS BETWEEN TABLES

When multiple tables are set up in Ms-Access, there should be a way of telling Access how to bring that information back together again. The first thing you need is to define relationships between your tables. Then create queries, forms and reports to display information from several tables at once.

A relationship usually works by matching data in key fields (usually a field with the same name in both tables). These matching fields are usually the primary key from one table, which provides a unique identifier for each record, and a foreign key in the other table. For instance, lecturers can be associated with the students they are responsible for by creating a relationship between the lecturer's table and student's table using the **Staff_ID** field.

If the above criteria are met, the following steps are required to create relationship between tables.

- Step1** Click on **Tool**
- Step2** Select **Relationship** at the top of the database Window view.
- Step3** Select the table you want to link together, by clicking on them.
- Step4** Select the **Add Button**.
- Step5** Drag the primary key of the parent table (i.e. Lecturer's table), and drop it into the same field in the child table (i.e. Student's table).
- Step6** Select Enforce **Referential Integrity**.
 - a. When the **Cascade Update Related Fields** check box is set, changing a primary key value in the primary table automatically updates the matching value in all related records.
 - b. When the **Cascade Delete Related Records** check box is set, deleting a record in the primary table deletes any related records in the related table.
- Step7** Click **Create** and **Save the relationship**

QUERIES IN MS-ACCESS

A query is a way to analyze, select and ask questions about data that are stored in table. Query can be used to locate record in tables, perform calculations and total or select certain kinds of data. One of the benefits of working with a relational database is that instead of sorting all of your data in one table, you can separate data into several tables and join the table when you need combine data.

EXERCISE 8

1. Ms- Access has been described as a genuine database application than other programs, discuss.
2. Write short notes on the following; table, query, form, report, module, and macro.
3. Write an algorithm to perform the following operations in Ms- Access;
 - a. Create a database without using the database wizard.
 - b. Open an existing database
 - c. Create a table using wizard.
4. What do you understand by key field? Give examples.

REFERENCES

- Adigun J. O (1999) Information Technology, Lagos, Lapteck publishers
- Agrawal R. and D. DeWitt. (1985). *Integrated Concurrency control and Recovery Mechanisms: Design and Performance Evaluation*. *ACM Transactions on Database Systems*, 10(4):529-564.
- Ajibade, L. S, Adaet D. B & Ikuborije, P .O (1999) Introduction to Computer Packages, Ilorin, Kola Success Publishers.
- Armstrong W. (1974). *Dependency Structures of Database Relationships*. In *Proc. IFIP Congress*.
- Ayeni J. O (1992) Fundamentals of Computing, Lagos, University of Lagos Press.
- Chen. P. P (1976). *The Entity-Relationship Model toward a Unified View of Data*. *ACM Transactions on Database Systems*, 1(1):9-36.
- Civelek F., A. Dogac, and S. Spaccapietra. (1988). *An Expert System Approach to View Definition and Integration*. In *Proc. Entity Relationship Conference*.
- Codd, E.F. (1970). "A Relational Model of Data for Large Shared Data Banks". *Communications of the ACM* 13 (6): 377–387.
- Codd. E. (1972). *Further Normalization of the Database Relational Model*. In *Data Base Systems*. ed. R. Rustin, Prentice Hall.
- Fagin, R. (1979). *Normal Forms and Relational Database Operators*. In *Proc. ACM SIGMOD Conf on the Management of Data*.
- Huang J., J. Stankovic, K. Ramarnritham, and D. Towsley (1991). Experimental Evaluation of Real-time Optimistic Concurrency control Schemes. In *Proc. Intl. Conf. on Very Large Databases*.
- O'Brien, James A. (2003). Introduction to Information Systems, McGraw-Hill, 11th Edition
- Peter Bishop (1987) Computing Science, Hong Kong, Thomas Nelson
- Richard W.B & Jeffrey M.D (1986) Using computer in an information age, Delmar Publisher Inc.
- Simon, C. G, Witt G C. (2005). *Data Modelling Essentials*. (3rd ed), San Francisco: Morgan Kaufmann.
- Wiederhold G. (1983). *Database Design* (2nd ed.). McGraw-Hill.
- Winslett. M. S (1988). *A Model-based Approach to Updating Databases with Incomplete Information*. *ACM Transactions on Database Systems*, 13(2):167-196.
- [http:// Microsoft.com/office/access/default. htm](http://Microsoft.com/office/access/default.htm)
- <http:// Microsoft.com/office/Excel/default. htm>